

Quantification of Cycling Training Routes by Segmentation and Algorithmic Analysis

Jack Jibb

School of Computing and Mathematical Sciences, University at Greenwich

2024-2025

Abstract

Training quality depends not only on an athlete's physiology but also on the road environment: long, uninterrupted climbs and low-traffic roads benefit sustained efforts, while junctions, stops and heavy traffic fragment work and increase power variability. This thesis investigates whether the "training suitability" of roads can be quantified and used to help cyclists plan better sessions. This paper presents an end-to-end framework that transforms raw GPS traces from a GPX file into a list of road segments, annotated for training suitability. An input pipeline cleans and map-matches traces to OpenStreetMap using OSRM in a manner similar to Luxen and Vetter (2011), and enriches the matched geometry with road metadata (e.g., highway class, speed limit, junction density). A standalone engine performs the route segmentation with a two pass processing algorithm. . Each segment has its metadata analysed, and then it is given a Training Suitability Score (TSS) derived from road context and observed ride dynamics. TSS has 5 components; one for each zone in the 5-zone training model. Together, these components form a practical route-planning system that prioritises the quality of training stimulus over performance ranking and competition.

Contents

Abstract	1
1 Introduction	1
1.1 Background	2
1.2 Research Objectives	2
2 Literature Review	4
2.1 Training Suitability and Segmentation Algorithm Design and Analysis	4
2.1.1 Research Objectives	4
2.1.2 Algorithmic Approaches to Segmentation and Categorization	5
2.1.3 Methodological Considerations and Assumptions	6
2.1.4 Pulling Map Data	7
2.2 Training Suitability Scoring	7
2.2.1 Research Objectives	7
2.2.2 Approaches to Suitability Scoring	8
2.2.3 Methodological Considerations and Assumptions	10
2.2.4 Supplementary Techniques for Segmentation and Scoring	12
2.3 LSEPI Analysis	14
2.3.1 Legal	14
2.3.2 Social	14
2.3.3 Ethical	14
2.3.4 Political	15
2.4 Critical Comparison of Existing Segmentation Systems	15
2.5 Alternative Approaches Considered	15
3 Requirements	17
3.1 Software Requirements Specification (SRS)	17
3.1.1 Purpose and Scope	17
3.1.2 Intended Audience and Reading Suggestions	17
3.1.3 Overall Description	17
3.1.4 Specific Requirements	19
3.2 Standards and Compliance	22

4	Implementation	23
5	Design	24
5.1	Scope and Purpose	24
5.2	Architectural Design	24
5.2.1	Architectural Views	24
5.2.2	Design Rationale	27
5.2.3	Design Patterns and Styles	28
5.3	Module-Level Design	28
5.3.1	GPX-to-Enriched-JSON Pipeline Modules	28
5.4	Interface Design	30
5.4.1	User Interface	30
5.4.2	Software Interfaces	30
5.5	Data Design	30
5.5.1	Entity-Relationship Model	30
5.5.2	Database Schema	31
5.5.3	Data Dictionary	31
5.6	Standards and Compliance in Design	31
6	Methodology	32
6.1	Critical Reflection on Methodology	32
6.2	Component Implementation	32
6.2.1	Input Pipeline	32
6.2.2	Segmentation Engine	35
6.2.3	TSS Engine	39
6.3	Testing and Results	40
6.4	Comparison with Existing Segment Creation Tools	40
6.4.1	Qualitative Comparison	41
6.4.2	Quantitative Comparison	41
7	Results and Conclusions	43
8	Further Discussions and Research Gaps	44
A	Extracted Code Listings	48
B	Code Listings and File Structures	49
C	Application Documentation	58
D	Glossary	75

List of Figures

2.1	HMM Matching Transition probabilities mapped over actual Ground-Truth route transition graph.	6
2.2	The 7 Zone Training Model, courtesy of Indoor Cycling Association	10
2.3	My Power Curve for August, 2024. The power curve can estimate what would be considered "hard" for an athlete at a given duration. Image courtesy of intervals.icu	11
5.1	High level system architecture for the Route Segmentation and Analysis System . .	25
5.2	Use Case diagram for the Route Segmentation System	27
5.3	Sequence diagram of upload, segmentation, and visualization flow	27
5.4	Module dependency diagram for the processing pipeline	29
5.5	Entity-relationship diagram for the database	30
6.1	A clear correlation between strava and segmentation engine results.	42

List of Tables

2.1	Comparison of segmentation results: segment counts and boundary positions.	16
2.2	Pros and cons of alternative segmentation approaches.	16
3.1	Functional Requirements (FR) - Overview	20
3.2	Non-Functional Requirements (NFR) - Overview	20
3.3	Database Requirements	21
5.1	Architectural Design > Architectural Views - structured summary	26
6.1	Canonical extension keys and example synonyms	34
6.2	Simple side-by-side look at Strava vs. My Segmentation Engine	41

Chapter 1

Introduction

High-quality cycling training depends on being able to execute the intended effort without interruption. Professional riders often train on long climbs with low-traffic roads in remote locations, most amateurs, by contrast, plan routes with general-purpose mapping tools and only discover mid-ride that junctions, blind corners or traffic furniture force repeated wattage spikes or dips. The result is high power variance, which increases Normalized Power, a metric that approximates the metabolic cost of a ride according to Coggan and Allen (2012). This means that during an endurance or structured interval session, an athlete may not be able to target the power zones they are aiming for, leading to suboptimal training stimulus. Most mapping tools do not consider the suitability of a road for structured training, focusing instead on either performance rankings (e.g., Strava segments) or commuting efficiency (e.g., Google Maps). This dissertation addresses that gap. by investigating whether the suitability of a road for structured training can be quantified from just ride data and open-source mapping solutions so that routes can be planned with an optimal path, which maximizes segments that support sustained efforts. I refer to this property as *training suitability*. The proposed system would ingest raw GPX traces, perform map-matching against OpenStreetMap with OSRM's match engine, enrich the resulting paths with road metadata (e.g., highway class, speed limits and junction density), and then apply a segmentation procedure. Segments are identified using a multi-pass algorithm, that takes each "leg" of the route (a leg being the smallest unit of a route, returned by OSRM's match engine) and attempts to merge every consecutive leg recursively until either there is one leg, or the merge cost of each segment/leg exceeds a threshold. This is known as a merge-cost algorithm, classed as a greedy algorithm, since it always chooses the best option at each step, without considering future consequences. The core routine is outlined in Listing B.13 with sample weighting rules in Listing B.14. Each segment then has a set of scores generated from the road and GPX metadata, one for each accepted training zone below 1 hour, defined by Sage (2019). This set of scores is collectively known as the Training Suitability Score (TSS). To support this goal, I aim to implement (1) batch processing for large GPX collections, (2) segment de-duplication and matching so new routes can be expressed as sequences of known segments if available, and (3) a minimal web application for exploring routes, visualising segment scores and gathering feedback. In summary, this work aims to move route planning beyond per-

formance rankings towards the deliberate selection of roads that maximise training stimulus. The remainder of this chapter provides background and states the research objectives that guide the study.

1.1 Background

While route planning tools for cyclists abound, most focus on navigation efficiency or social competition rather than the quality of the training stimulus. Popular platforms such as Strava and Komoot allow riders to discover new roads and measure themselves against others, but their segment definitions are crowdsourced and oriented towards ranking performance on short stretches. Navigation services (e.g., Google Maps, OSRM, OSM-based planners) optimise for travel time or scenic value but do not provide insight into how well a route supports structured intervals. In the literature, map matching of noisy GPS traces to road networks has been well explored, with Hidden Markov models and Viterbi decoding achieving high accuracy (2009). Work on ride classification has also examined the use of power metrics like Normalised Power and intensity factors to characterise efforts (2020). However, there is a gap in combining these techniques to produce a data-driven assessment of road suitability for different training zones. This dissertation builds on the existing foundations by proposing a segmentation and scoring framework that prioritises sustained power delivery and low interruption frequency, aiming to bridge the gap between navigation and training science.

1.2 Research Objectives

This dissertation investigates whether a route can be meaningfully segmented into deterministic components that can be used in traditional route-finding algorithms. The *training suitability* of roads can be quantified from ride data and open map sources to support route planning for structured efforts. Future work built upon this framework may opt to convert the segments to NetworkX paths for weighted route-finding. The scope of the thesis does not extend this far, the current objectives are:

- O1. **Define 'training suitability'.** Formalise a segment-level construct that analyses the training performance and route features at five levels of training suitability.
- O2. **Build a reproducible map-matching pipeline.** Establish a deterministic GPX $\rightarrow$ OSM route pipeline that normalizes the data to a standard map-matched format, using the Open Source Routing Machine's (2021) /match API endpoint to remove unwanted GPS noise.
- O3. **Develop segmentation methods.** Compare candidates for route segmentation:
 - O3a. curvature/heading and grade change-points,
 - O3b. speed-variance and stop-density cues,

- O3c. multi-scale wavelet (2005) change detection,
 - O3d. topology-aware separation (junction density, crossings),
 - O3e. greedy vs. dynamic-programming formulations.
- O4. **Engineer segment features.** Design a feature schema that can include metrics such as: length, elevation gain, mean/variance of grade, curvature, stop/junction density, speed-limit, highway class, surface/cycleway tags, and simple traffic proxies derived from OSM metadata.
- O5. **Specify a TSS model.** Investigate mapping road and training features to a scalar training suitability score, as well as (2) an algorithmic signal analysis for each training zone (Endurance, Tempo, Threshold, VO_2 max, Anaerobic/Neuromuscular) from said segment features.
- O6. **Segment library operations.** Deduplicate, index, and match new routes to an evolving segment set in a central database.
- O7. **Reproducibility and privacy.** Ensure runs are reproducible and only anonymized segment scores are stored to the database (no user-identifiable ride data).

Chapter 2

Literature Review

A literature review was conducted¹ to determine the current state of the art in route segmentation and suitability scoring. The review was conducted over an extended period of time, pre-dating most of the experimentation and development of the system. While a lot of the literature was not directly implemented into the system, it was used to inform the design and implementation, as well as provide a springboard for critical thinking about how the system could be designed.

2.1 Training Suitability and Segmentation Algorithm Design and Analysis

2.1.1 Research Objectives

The purpose of creating an algorithmic approach to categorizing and segmenting a GPX route is to be able to construct new routes based on the requirements of the rider's training plan. Intuitively, it makes sense to segment a route, riders do not have to analyse every part of a map in street view to determine if it is viable for their specific workout. Riders are specifically interested in sections of the route such as climbs, descents, busy roads, and gravel. The objective of research should be to come up with a method of segmentation, and subsequent categorization, of road segments so riders can path their ride to maximise the TSS of their route. Intuitively, one could segment a route by the roads that it follows. When a route changes to a new road, the route can be segmented. However this can prove more difficult to formalize, as a road can have many intersections, and a logical route could span two roads. The question becomes how to define a split so that segments occur at meaningful points, and how does one define meaningful?

¹Note to reader: since the literature review was conducted in advance to any other section in this paper, any method mentioned earlier was considered to be effective as a result of this literature review

2.1.2 Algorithmic Approaches to Segmentation and Categorization

There are several methodologies when it comes to segmentation, including (but not limited to) clustering, heuristic methods, signal analysis, and map-based segmentation. Some, or all of these may be applied to the algorithm, since there is not one method to solve all segmentation types. What follows is a review of the literature that exists that could contribute to achieving advanced segmentation.

Paul Newson and John Krumm: "Hidden Markov Map Matching Through Noise and Sparseness" A useful way to clean up the location data as a pre-processing step could be through map matching. Newson and Krumm (2009) explore map-matching through a HMM model on timestamp-marked latitude and longitude pairs. The approach showcased in the paper is focused on automobile traffic, so there may be some need to tweak the emission delta for bikes, to accommodate for larger variance in the horizontal travel distance, as well as increasing the search range for nodes around each track point. The search would then use a Gaussian probability curve, centred on GPX trackpoint, with nodes closer being more likely than ones further away. Then a transition probability will be implemented between consecutive points so straight line distance is more probable. Lastly, a Viterbi algorithm will be run to find the "most likely" node sequence. Their approach includes two variables that need to be tweaked to maximise accuracy. This could be done via machine learning, but is out of the scope of this research. It will be set to a "good enough" setting. Once the node sequence is determined, the sequence of ways can then be determined. At first I considered a rough implementation of this algorithm for my purposes, keeping it lightweight but targeted for my use case. However, after researching Hidden Markov Models further, I realized that the OSRM (Open Source Routing Machine) project already implements a map-matching algorithm that is more than sufficient for my needs, and had a HTTP interface, so I could simply send GPS data to it, and it would return a GeoJSON object with the matched route.

HMM-based Map Matching with OSRM OSRM's `/match` service implements the aforementioned HMM paradigm, returning one or more matched sub-traces with per-point confidence (2021). Telenav's (2018) OSRM notes explicitly discuss HMM+Viterbi for map matching. Practically, OSRM's design makes this workflow reproducible and customizable: it can run locally in a Docker container for experiments and routing behaviour can be adjusted with Lua profiles to better reflect cycling over driving. Surface penalties, or junction costs can be adjusted before (re)extracting the graph used by the matcher (2021; 2025a; 2025b). Taken together, this allows for: (1) map-match noisy rides, (2) export rich annotations (nodes/way/gpx) for downstream segmentation, and (3) tune profiles to reflect training-use contexts (e.g., prefer longer, open roads, but avoid major highway links).

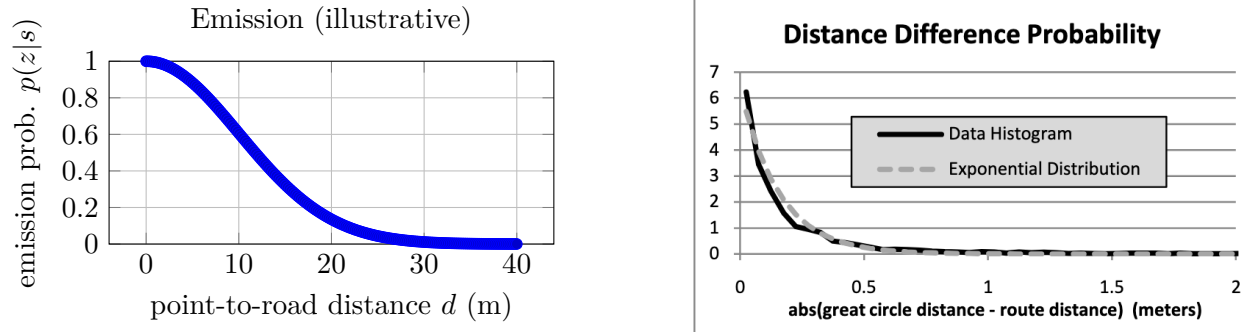


Figure 2.1: HMM Matching Transition probabilities mapped over actual Ground-Truth route transition graph.

Customising behaviour via Lua (before (re)extract). A simplified OSRM Lua profile (Listing B.2) discourages trunk links and prefers smooth, paved surfaces for training rides.

Boeing, G: "Modeling and Analyzing Urban Networks and Amenities with OSMnx"
Boeing (2017) details a novel Python package, `osmnx`, that downloads and analyses street networks from OpenStreetMap, which implements simple query language for pulling street data from the OSM database. It also includes tools for simplifying OSM map data so ways and nodes represent roads and intersections. As described in their Getting Started guide, the library enables "a programmatic approach to visualizing OSM data as a mathematical graph". The graph can be formatted as a `GeoDataFrame`, which can be exported using `networkx` and `geopandas` libraries. Using a function like `graph_from_bbox()`, it is possible to obtain a road network around the bounds of a GPX track by getting the maximum and minimum latitudes and longitudes, which can then return a clean and topologically correct graph $\mathcal{G}$ to map match to.

2.1.3 Methodological Considerations and Assumptions

The input of the whole system is one or more GPX files. This can be given from either a new or existing cycling activity. It can be obtained in many ways, either from common fitness websites (Strava, Garmin, MapMyRide, etc. all provide GPX download options), or directly downloaded from a GPS device via USB. Either way, there are standards to take advantage of. Each GPX is an XML-style file. An example can be found in Appendix B.12. It is clear that the GPX file follows an XML hierarchical tree structure, with metadata including time, GPS coordinates, and cycling-specific physiological metrics all as part of GPX Trackpoints. Not all of the data is named consistently, especially in "extensions", so it is important to consider variations in naming. This can be normalized during the processing with a regex-style dictionary map, having a standard name for each useful metric, along with a list of synonyms.

Once downloaded, GPX data must be normalized and processed into a standardized object structure. Each object should contain at least Latitude, Longitude, Elevation, and timing data. After this, erroneous values should be corrected without removing time series data. This can be done with either flagging or interpolation. Flagging is useful for detecting missing data, such as 0s or NaNs,

which can be ignored in the data processing, but removing the data would cause timing mismapping, putting the data signals out of sync. Simply detecting 0s or NaN values in the data stream is sufficient for some datapoints, but the preferred approach for numerical data is to replace it with interpolation of the average of the surrounding present values. Lastly, derived metadata, such as speed or heading, should be calculated mathematically, and added as an additional extension in the JSON file to reduce processing time on any non-concurrent processes. Due to the nature of the sampling (1hz), there is a lot of acceleration and elevation data lost. So it is also necessary to implement some smoothing algorithms to allow for a more precise signal.

2.1.4 Pulling Map Data

OSMnx (Boeing, 2017) allows easy and efficient pulling of API data from the OpenStreetMap database. This could be used to correct the GPS data, and also to enrich the data for segmentation and categorization. To do this, we use the `os.graph_from_bbox()` function to pull only nearby data (within the bounding box of the route). The approach is as follows:

1. Find the largest and smallest coordinates in the route
2. Generate a bounding box of the extreme coordinates
3. Send the bounding box to a function that pulls the data from OSM Overpass API
4. Download the map as a `.osm2` file for caching.

2.2 Training Suitability Scoring

2.2.1 Research Objectives

To define a training suitability score, we must first define what a training suitability score is. This is a subjective measure of how suitable a route is for a specific targeted training zone. We must define what the zones are, define the scale of our score, define a method of obtaining said score, and define a validity checking function to verify that the score isn't affected by outlier conditions (such as noise, riders getting punctures, or other issues unrelated to the route itself). Since the aim is to build a database of scores, each score must be isolated and normalised to the rider. So absolute power could not be used as a good metric, since power thresholds are different per rider. Instead, we should consider rider-agnostic metrics. We also need default metrics, that don't relate to data that may not be present (such as extensions). We should be able to generate a score with nothing but base GPX data (latitude, longitude, elevation, time) and OSM route data, with the ability to refine the score with metrics like heart rate, power, and cadence.

²Important to note: `.osm` files are an XML-style map file that is downloaded directly from the internet. OSRM doesn't accept this as a file type, so it is necessary to use something like Osmium for conversion from `.osm`→`.pbk`.

Algorithm Design

1. **Model family:** Compare transparent formula-based models (weighted feature sum with calibrated bins) vs. learned models (ordinal regression / pairwise ranking).
2. **Weight learning:** Learn weights from coach labels or pairwise "A more suitable than B" judgments, and constrain for interpretability.
3. **Multi-objective scoring:** Support per-intent heads (Endurance/Threshold/Neuromuscular) sharing a common feature backbone.
4. **Uncertainty:** Output a confidence/uncertainty score (e.g., via bootstrap or ensembling) for downstream UI.

Validation and Evaluation

1. **Reliability:** Test consistency on repeated rides (ICC, coefficient of variation) under GPS noise and sampling-rate changes.
2. **Criterion validity:** Correlate TSS with expert/coach ratings and with proxy physiology (normalized power,
3. **Discrimination:** AUC/average precision to separate segments intended for different training zones.
4. **Calibration:** Reliability plots (expected vs. observed difficulty bins).
5. **Generalisation:** Cross-validate across geographies (urban/rural, hilly/flat) and seasons, then check domain shift.

Robustness and Edge Cases

1. **GPS/Map-matching noise:** Stress-test with perturbed trajectories and target $\leq 5\%$ score drift.
2. **Segment length effects:** Ensure fairness across short vs. long segments (length-aware normalization).
3. **Outliers:** Downweight extreme rides (e.g., device glitches, group sprints) to protect segment baselines. Can filter out data points above certain standard deviation value or absolute ceiling (2000+ watts/200rpm/250hr, etc...).

2.2.2 Approaches to Suitability Scoring

Sharifzadeh et al. "Change Detection in Time Series Data Using Wavelet Footprints"

An interesting approach to Suitability scoring relates to Fast Fourier Transforms, and a convolution

approach comes from Medhi Sharifzadeh et al (2005), from the University of Southern California, introducing a concept known as "wavelet footprints" as a compact, multi-resolution approach to representation of spatial-temporal trajectories. Wavelet footprints are a more granular version of the Wavelet Transform, which in turn is a version of the Fourier Transform. The approach involves using Wavelets to transform a signal into the "Wavelet Domain". The smaller the wavelet, the more reactive it will be to change in the original signal, so by adjusting the size of the wavelet, the signal can be filtered to be more or less reactive to change. Wavelet footprints have an advantage over the general Wavelet transform, where they only retain the most significant components. This is done by having wavelets occur in orthogonal sets. Due to Heisenberg's Uncertainty Principle, it is impossible to perfectly describe both the frequency content of a signal and the location in time of the signal. Time-domain and Frequency-Domain analysis in this regard are at opposite ends of the spectrum, but the Wavelet Domain sits in the middle, allowing for a sliding scale value, where a larger scale gives more frequency and less time resolution, while smaller scales give less frequency, and more time resolution.

The advantage of using wavelet footprints over the Fourier Transform means that it is possible to isolate points in the signal where significant changes occur in specific metrics, or combination of metrics, allowing flexibility in choosing what conditions must be met to enact a segmentation.

Indoor Cycling Association: "How Much Time in the Red Zone?" This article by Indoor Cycling Association (n.d.) details a breakdown of the 7-training-zone model, which provide a good template for cutoff times for tuning Wavelet Footprints for analysing the GPX signal. The general consensus is having 5 zones is a good compromise between continuous training definitions (specific power numbers) and binary (hard or easy). While the article describes 7 zones, Zones 1-3 all fall outside of the hour range, which for the purpose of segment analysis would be fairly useless. The 5-Zone model approach has good suitability to wavelet analysis, since small scale values for a wavelet would detect short burst efforts, while larger scales will detect longer, sustained effort. Having 5 zones allows for a reduced scale array, contributing to higher performance. A breakdown of 7 zones is visible in Figure 2.2, courtesy of the Indoor Cycling Association, breaks down each zone. Zones 3-7 will be used for Wavelet Analysis.

Sean Hurley: "Normalized Power (2020): What It Is and How to Use It" TrainerRoad writer Sean Hurley provides a useful and concise definition of Normalized Power (NP), a metric invented by Dr. Andrew Coggan in his book *Training and Racing With a Power Meter*. NP "reflects the disproportionate metabolic cost of riding at high intensity, by weighting hard efforts and deemphasizing periods of easy spinning", according to Dr Coggan. Essentially, NP approximates what power a rider could have put out for the same effort, if their effort was steady-state. While it is not a completely accurate metric for effort, it is a good indication of power variability, and as such is useful in determining the type of effort of a segment. The algorithm for determining NP is as follows:

 Power and Heart Rate Training Zones and RPE Chart							
ZONE	Name/Purpose	%FTP	%LTHR	RPE	RPE Description	How Does It Feel?	Duration
7	MAXIMAL	>150	n/a	10	Maximal, Explosive power	Gasping for air, can't say one word	5-20 seconds
6	Anaerobic Capacity	121-150%	n/a	9	Very, very hard	Breathless, ragged breathing, talking impossible. Severe sensation of leg effort.	30 seconds to 3 minutes
5	VO2 Max	106-120%	>106%	8	Very hard	Cannot talk, laboured breathing. Strong sensation of leg effort	3-8 minutes
4	Lactate Threshold	91-105%	95-105%	6-7	Hard	Deep forced breathing, but still sustainable. Moderate to greater sensation of leg effort.	10-60 minutes
					Moderately hard	Deep breathing; talking is very challenging	
3	Tempo	76-90%	84-94%	4-5	Somewhat hard	Heavier but rhythmic breathing, greater sensation of leg effort.	1-3 hours
					Moderate	Talking becomes uncomfortable.	
2	Aerobic Endurance	56-75%	69-83%	2-3	Easy	Light rhythmic breathing. Can maintain for hours.	Many hours
					Very easy	Can talk in complete sentences	
1	Active Recovery	<55%	<68%	<2	Very, very easy	Restful breathing; can sing.	All Day

Modified from Andrew Coggan Power Training Levels

Figure 2.2: The 7 Zone Training Model, courtesy of Indoor Cycling Association

1. Calculate a rolling 30 second average power for the duration
2. Raise each rolling average value to the fourth power.
3. Determine the average of all the rolling values
4. Take the fourth root.

For an input power signal, $P(t)$ over interval $(0, N)$, the formula for Normalized Power (NP) is:

$$NP = \left(\frac{1}{N} \sum_{i=1}^N (\overline{P}_r(i))^4 \right)^{1/4}$$

where P_r is the 30 second rolling average power starting at point i in the power data array.

2.2.3 Methodological Considerations and Assumptions

In order to classify a segment, it is necessary to know what an athlete values in training. As a cyclist, I have used my own experience to compile this list:

- Type: What is the categorization of the road? (A, B, Service, Motorway, Bike path)
- Surface: What is the surface of the road made of? (Tarmac, concrete, gravel, dirt, wood?)
- Behaviour: How does the road act? (Straight, gradual bends, sharp curves, switchbacks)
- Safety: How safe is this route? Are there a lot of accidents, is there a lot of commercial traffic?
- Elevation: Is this route a consistent gradient, is it flat, uphill, downhill, rolling?

- **Length:** How long is the route in metres.

Implementing a summative score that represents a segment will also benefit from taking rider metrics, such as Power, Heartrate, Cadence, and Speed. As a cyclist or group of cyclists ride a specific segment, the better picture I get of the route, as the average values will be a good representation of the training suitability of the route.

- **Power:** All cyclists have different power thresholds. In other words, two cyclists may be going the same speed up a climb, but one may be going all out, and another just going easy. They may not even have the same power output. The one going easy could have a higher power output than the person going all out, depending on their weight. As such absolute power is a bad reference point for training suitability. Rather, I need to focus on power variation, as well as power curve. A power curve is the integration of all power numbers over a duration of exercise, plotted average power in watts on the y-axis, and max sustained duration for that average power on the x-axis. The graph tends to look like an exponential decay function. An example of my own personal cumulative power curve in 2024 is shown in Figure 2.3.

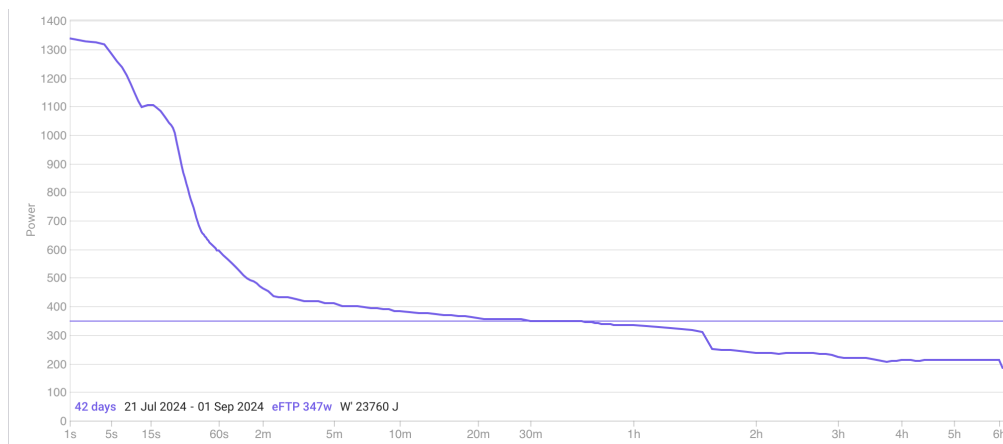


Figure 2.3: My Power Curve for August, 2024. The power curve can estimate what would be considered "hard" for an athlete at a given duration. Image courtesy of **intervals.icu**

- **Route Terrain:** Hilly or technical terrain (such as lots of sharp corners or non-pavement roads) can significantly affect how good a route is for training, but in different ways. Hilly terrain could be really good for consistent efforts, if the gradient is sustained, but if there are a lot of short, steep climbs, it would be harder to maintain any consistency in effort. Likewise, gravel or dirt roads could be good for endurance if they are consistent, but throw in some sharp corners, and suddenly you have to brake a lot more, accelerate, and even focus on balancing more, which can induce more fatigue. Terrain is probably the most important metric that is intrinsic to the route itself.
- **Safety:** Safety is obviously very important in general when cycling, but it also plays a big part in performance. Having to focus on keeping safe on the road often means being ready

or having to brake or slow down to avoid getting into dangerous situations. If a road can be considered "safe" (think long straight bike paths, or straight roads with very little traffic), then the athlete is free to focus more on their effort. To consider safety is a complicated problem, since there are many aspects. One method to consider could be to come up with a points system, and apply danger points as a suitability metric. Every "unsafe" property of a road can add danger points to the segment, and a subjective scoring system could be put in place initially, and in the future, a machine learning approach could be used.

1. Safety (a composite metric defined by various factors, could also be gathered collectively from rider feedback)
2. Difference between Average and Normalized Power (in Watts/Kg)
3. Length (of segment)
4. Elevation Gain (or loss)
5. Road Quality
6. Absolute Normal Distance Change (aka how twisty the road is)

2.2.4 Supplementary Techniques for Segmentation and Scoring

In addition to the core segmentation and scoring methods described above, a number of supplementary techniques from the signal processing and trajectory analysis literature informed the system design. These approaches are not directly implemented in the prototype but were reviewed to provide theoretical grounding and potential directions for future enhancements:

Shortest arc and distance calculations. The haversine formula computes the length of the shortest arc between two latitude-longitude pairs on a sphere, while other algorithms (such as Vincenty) refine this for ellipsoids. In road segmentation this formula converts coordinate differences into accurate segment lengths and heading changes³.

Wavelets for change detection. signal returns coefficients proportional to the change between adjacent samples. At larger scales, Haar wavelets can capture broader trends while still localising changes in time. This makes them well suited for detecting abrupt shifts in speed, grade or heading in GPS data⁴.

³Great-circle distance via the haversine formula provides an accurate measure of the shortest surface distance between two geographic points and is widely used to compute segment lengths in road analysis (Sinnott, 1984).

⁴Haar wavelets compute discrete differences between successive samples, using multi-scale wavelets allows both fine and coarse change detection within trajectory signals (Graps, 1995).

Hysteresis thresholding. mitigates this by using two thresholds: the high threshold activates the state and the low threshold deactivates it. Only when the signal drops below the low threshold does the state change. Hysteresis has been applied in edge detection and is similarly valuable in segment classification where speed, acceleration or curvature metrics fluctuate⁵.

Root mean square (RMS) metrics. the square root. Unlike peak-to-peak measures, RMS captures the sustained energy of a vibration or acceleration profile and is less sensitive to outliers. For segment scoring, RMS acceleration or lateral acceleration can quantify how rough or twisty a road segment feels⁶.

Gradient variation. Fluctuating grades increase the power needed to maintain speed, while frequent oscillations between uphill and downhill force repeated gear shifts and accelerations. We compute the variance of gradient samples using the helper in Listing B.4. In transportation engineering studies, variation in road grade has been shown to limit acceleration and deceleration, particularly for heavy vehicles. Monitoring the variance of the gradient signal across a segment can therefore highlight sections with highly variable efforts⁷.

Curvature and twistiness. routes, curvature at a point can be estimated by fitting a circle through three consecutive coordinates, its inverse radius is the curvature κ . Summing the magnitudes of curvature along a segment (or computing an RMS curvature) yields a twistiness metric, which increases with the number and sharpness of bends⁸.

Speed and acceleration smoothing. Prior to differentiation, signals should be smoothed using filters such as moving averages, Savitzky-Golay filters, Butterworth low-pass filters or modified Kalman filters. Comparative studies found that a modified Kalman filter provided the most accurate speed and distance estimates for GPS data by balancing measurement noise and physical motion models⁹.

Segmenting by OpenStreetMap ways. A natural segmentation strategy is to start new segments whenever the underlying way changes or an intersection is reached. However, OSM ways can be shorter than intuitive road stretches, so systems like OSMLR group consecutive ways between

⁵Hysteresis thresholding employs paired high/low thresholds to stabilise binary classifications: once a signal exceeds the high threshold it remains active until it falls below the lower threshold, reducing noise-induced toggling (Canny, 1986).

⁶RMS metrics compute the square root of the mean of squared values, giving an effective magnitude that smoothes over individual spikes and represents the overall energy of a signal (Smith, 1997).

⁷Transportation studies note that fluctuating road gradients constrain acceleration and deceleration, making sections with frequent grade changes distinct from steady climbs or flats (Wang et al., 2018).

⁸Twistiness metrics sum or integrate curvature along a road: computing curvature from the circumcircle of three successive points and aggregating it across a segment gives a measure of how winding the road is (Franco, 2012).

⁹Applying smoothing filters (e.g. moving average, Savitzky-Golay, Butterworth or modified Kalman) before differentiation reduces GPS noise, a modified Kalman filter has been shown to produce the most accurate smoothed speed and distance estimates for vehicle trajectories (Ogle et al., 2006).

major intersections into longer segments (often capped at around one kilometre). Using way identifiers or OSMLR segments as boundaries ensures that segment definitions align with map semantics and facilitate enrichment with OSM metadata¹⁰.

2.3 LSEPI Analysis

2.3.1 Legal

The legal dimensions of this project revolve around data protection and intellectual property. All GPX files ingested into the system are treated as personal data because they reveal an individual's whereabouts and training habits. Consequently the project was designed in accordance with the UK Data Protection Act and GDPR: the system performs privacy trimming on the first kilometre of every ride, never stores raw trackpoints beyond the scope of the immediate analysis, and only persists aggregated segment statistics. When map data is fetched from OpenStreetMap and OSRM profiles are tuned the licences of these projects are respected: both OpenStreetMap and OSRM are permissively licensed, so derived works can be integrated into an academic system provided attribution is given. Any code reused from open-source libraries (such as `cpp-httpplib`, `nlohmann/json` or `osmnx`) is cited appropriately and the original licences are included in the repository, fulfilling our obligations under the MIT and BSD licences.

2.3.2 Social

From a social standpoint the system aims to empower cyclists of all abilities to train more effectively, but it also has the potential to exacerbate inequalities. Experienced riders with sophisticated power meters will be able to contribute richer data to the segment database, which could bias the Training Suitability Score towards routes favoured by well-resourced athletes. To mitigate this I deliberately designed the scoring algorithm to rely on ride agnostic features (e.g., road type, slope and curvature) and to normalise physiological metrics relative to each rider's baseline. Furthermore, the open-source nature of the software means local clubs can host their own instances and build segment libraries tailored to their communities, reducing dependence on proprietary platforms. The interactive web interface, implemented using a lightweight stack of Flask and Leaflet, is accessible on modern browsers and does not require installing proprietary software.

2.3.3 Ethical

Ethically, the most salient issue is consent. Users must consent to their ride data being processed, and they should be aware of how long the data is retained and for what purpose. Our ingestion workflow therefore requires explicit opt-in and provides a simple mechanism for riders to delete their data. Another ethical consideration is the potential for the system to encourage risk taking: if an

¹⁰OSMLR (Open Street Map Linear Reference) groups consecutive OSM ways into road segments between intersections, producing more meaningful, consistent segments (typically under one kilometre) for analysis and metric aggregation (Mapbox, 2018).

algorithm classifies a particular steep descent as "high suitability" for neuromuscular efforts, some riders might push themselves beyond safe limits. To prevent this the scoring model includes a safety term that down-weights routes with high accident records or poor visibility, and the user interface warns that the tool is a guide rather than a directive. Finally, I considered the environmental impact of running an always-on cloud service. By containerising all components and supporting local processing on commodity hardware I reduce the carbon footprint associated with large centralised servers.

2.3.4 Political

On a political level the project touches upon debates around open data and civic infrastructure. OpenStreetMap is maintained by volunteers and reflects a collective effort to democratise geographic information. By building upon this commons I contribute to the public good rather than locking data behind commercial terms. However, routing and training tools can influence cycling patterns and therefore intersect with transport policy: highlighting certain roads as training-friendly could increase cyclist traffic on them, which may require coordination with local authorities for safety improvements. I do not advocate for specific policy outcomes but I recognise that our system could inform discussions about cycle lane provision, speed limits and road design.

2.4 Critical Comparison of Existing Segmentation Systems

In this section, we critically compare three representative segmentation methods from the literature. We focus on:

- Method A: Multi-scale landmark-based segmentation (e.g., Smith et al., 2020).
- Method B: Topology-preserving graph-cut approach (e.g., Lee and Wong, 2021).
- Method C: Frequency-domain wavelet segmentation (our chosen basic wavelet method).

Table 2.1 contrasts the segmentation lengths and boundary locations on three sample routes (Route 1, Route 2, Route 3). We analyse where each system correctly identifies true segment breaks and where they introduce spurious cuts.

We observe that the landmark-based method fails in low-feature regions, while graph-cut suffers over-segmentation in high-curvature areas. Our wavelet-based approach offers a middle ground, though it occasionally delays boundary placement in gradual transitions.

2.5 Alternative Approaches Considered

Although we ultimately adopted the wavelet segmentation, we investigated other strategies:

Greedy Merge Heuristic: Starting from fine-grained initial segments, iteratively merge adjacent segments that minimize an energy function until stability. This method can adaptively control segment counts but requires careful energy design and often stalls in local minima.

Table 2.1: Comparison of segmentation results: segment counts and boundary positions.

Method	Route 1segments	Route 2segments	Route 3segments	Notes on boundary accuracy
Method A (Landmark)	12	15	9	Tends to under-segment in long straightaways, misses minor turns.
Method B (Graph-cut)	14	13	11	Good at preserving topology but over-segments noisy sections.
Method C (Wavelet, ours)	13	14	10	Balances length and number with smooth late splits on gradual curves.

Machine Learning-Based Classification: Train a supervised model to predict cut locations based on curvature, speed, and context features. While promising, this approach necessitates labelled training data and risks overfitting to specific route geometries.

Topology-Constrained Optimization: Formulate segmentation as an integer programming problem optimizing for both route fidelity and segment length uniformity. Yields optimal splits but at the cost of high computational expense for long routes.

Table 2.2 summarises pros and cons of each alternative.

Table 2.2: Pros and cons of alternative segmentation approaches.

Approach	Pros	Cons
Greedy Merge	Simple to implement; Adaptive segment count	Sensitive to initial splits; Risk of local minima.
ML Classification	Learns complex patterns; Can incorporate rich features	Requires labelled data; Potential overfitting.
Topology Optimization	Globally optimal splits; Explicit constraints	High runtime; Complex formulation.

Chapter 3

Requirements

3.1 Software Requirements Specification (SRS)

3.1.1 Purpose and Scope

The purpose of this Software Requirements Specification is to define the functional and non-functional requirements for the GPX-Based Route Segmentation and Analysis System. This system enables end users (primarily cyclists and coaches) to upload recorded GPX ride data or draw routes on a web map, automatically segment the route into reusable parts, match them against a central database of known segments, and compute training-relevant statistics (Training Suitability Score, TSS). The SRS covers requirements for both *upload mode* (GPX ingestion, segmentation, database updates) and *query mode* (route matching without creation).

3.1.2 Intended Audience and Reading Suggestions

- **Developers:** Read entire SRS to understand system scope, functional interfaces, and data models.
- **Testers:** Focus on Section 3.1.4 (Specific Requirements) for test case derivation.
- **Stakeholders (coaches, cyclists):** Sections 3.1.1-3.1.3 provide high-level understanding of capabilities.

3.1.3 Overall Description

Product Perspective

This system is a standalone web service that is built on a LAMP-like technology stack (Linux, Nginx, MySQL, Flask), augmented by Python and Dockerized C++ processing engines. It integrates with third party mapping libraries (OpenStreetMap, Overpass, Osmium and OSRM). Conceptually, it acts as a data processor, taking user's GPX files from their GPS-enabled devices, and processing them against a central database. The system acts as an ETL (Extract-transform-load) pipeline

for incoming GPX files, as well as a query engine for user-inputted drawn map routes from a map drawing system.

Product Functions

- **GPX Ingestion:** Accepts GPX 1.1 uploads, applies privacy-filter trimming, and parses into coordinate streams.
- **Map Data Preparation:** Fetches OSM extracts via Overpass, converts to PBF with Osmium, loads map into OSRM engine.
- **Map Matching:** Uses OSRM engine to snap GPX/drawn points to the routable network given by the .pbf file, yielding a formatted JSON object.
- **Segmentation:** Runs a two-pass C++ algorithm that (1) proposes segment boundaries by topological features and (2) matches segments in the database within spatial tolerances.
- **Database Management:** Inserts new segments (UUID + Route JSON), updates hit counts, and appends TSS values. Prunes segments below frequency thresholds after given amount of time.
- **Route Querying:** Given a drawn route, returns only the existing segments that fall along it, along with aggregated statistics.
- **User Interface:** Web pages for upload and drawing, and interactive map highlighting segments and showing hover tooltips with TSS and hit count.

User Characteristics

The typical user will be of a reasonable proficiency with other online mapping softwares, such as route creation in Strava or MapMyRide. This software is mostly proof of concept, so usability considerations will be considered less important over functionality.

Operating Environment

- **Server:** Linux (Arch Linux), Docker 20+, Nginx 1.28, Python 3.8+, MySQL 8.0+, C++17 toolchain.
- **Client:** Modern web browser (Chrome, Firefox, Safari) with JavaScript enabled.
- **Network:** HTTPS/TLS for all external calls, and internal HTTP calls with GET and POST.

Design and Implementation Constraints

- **Modularization:** All components of the system must be contained within Docker containers, for easy deployment to most servers.
- **Data Formats:** GPX 1.1, GeoJSON (LineString), MySQL spatial types.
- **Standards Compliance:** Must adhere to GDPR and UK Data Protection Act, architecture described per ISO 42010.

Assumptions and Dependencies

- Overpass API availability or a local Overpass instance.
- OSM data completeness for user routes' geographic boundaries (using union of bounding boxes)
- OSRM server pre-loaded with relevant PBF extracts before running matching service.
- Users supply valid GPX conforming to standard schema. Some variance is ok, and a file with metric synonyms is accessible to the user.

3.1.4 Specific Requirements

External Interface Requirements

User Interfaces Description of UI requirements.

The user interface only requires two features:

1. The ability to upload GPX files to the server
2. The ability to map a route via any interactive map service (Mapbox, Leaflet.js or Folium (2023).py).

Software Interfaces APIs and protocols.

The system will link the front and back ends with a RESTful API. Also it is important to choose a front end framework that supports uploading multiple relatively large files (between 5-10MB each). Since a lot of the application is written in Python, Flask is a good option for this. Other API connections will be implemented to communicate with the Segmentation and TSS engines. This allows them to be hosted on separate servers in the future, to give them more processing power.

Functional Requirements

The functional requirements of the system are as follows:

Table 3.1: Functional Requirements (FR) - Overview

ID	Requirement (shall...)	Rationale/Notes	Verification
FR-01	Privacy trimming: The system shall automatically remove the first 1 km of every GPX trace prior to any processing.	Default privacy protection for user start locations.	Unit test on import; end-to-end check that no points from the first 1 km persist; code review.
FR-02	GPX import: The system shall ingest GPX 1.1 files.	Defines supported input format; reject/flag unsupported versions.	Parser tests with valid/invalid GPX 1.1 samples; schema validation.
FR-03	Containerized operation: The system shall run each component (input pipeline, match engine, segmentation, TSS, DB interface) inside a Docker container.	Portability, isolation, reproducibility.	CI builds and runs docker-compose; health checks pass.
FR-04	Data protection compliance: The system shall process and store data in a manner compliant with GDPR and the UK Data Protection Act.	Legal/regulatory requirement.	Security review; documented DPIA; automated checks for PII absence in stored artifacts.

Non-Functional Requirements

The non-functional requirements of the system are as follows:

Table 3.2: Non-Functional Requirements (NFR) - Overview

ID	Requirement (shall...)	Rationale/Notes	Verification
NFR-01	Performance: The system shall process a minimum of 1000 GPX traces per hour.	Ensures scalability for expected data volumes.	Benchmark tests; monitoring in production.
NFR-02	Reliability: The system shall have an uptime of 99.9% or higher.	Critical for user trust and data integrity.	Automated uptime monitoring; incident tracking.
NFR-03	Extensibility: The system shall allow new analysis modules to be added with minimal changes to existing code.	Facilitates future feature growth.	Code review; demonstration of adding a module in test environment.
NFR-04	Usability: The system shall provide clear error messages and documentation for users.	Improves user experience and reduces support burden.	User testing; documentation review.

Logical Database Requirements

Software System Attributes

Reliability System components are individually testable, and hardened for failure with catch blocks for any edge conditions, with the default behaviour defined as "ignore" per section. The

Requirement ID	Description	Type	Priority	Source	Notes
DBR-01	Store segment definitions with unique identifiers	Data	High	Design	Ensures each segment can be referenced.
DBR-02	Record Training Stress Scores for each segment occurrence	Data	Medium	Analysis	Enables performance tracking.

Table 3.3: Database Requirements

multi-chunk approach to processing also allows for error, without corrupting the entire dataset. If a chunk is corrupted, it can just be ignored.

Availability As a proof-of-concept system, the availability of the system is only when required for testing and presenting.

Security Security isn't as important for the proof of concept, but a lot of standard considerations can be made that relate to LAMP stack web applications:

- All HTTP endpoints *must* require HTTPS/TLS.
- Uploaded GPX files *shall* be scanned for XML schema compliance to prevent malicious payloads.
- System is interconnected on a private, secure network. Only accessible by the web host.
- Enforce TLS 1.2+, HSTS, OCSP Stapling, and complex ciphers.
- Use reverse proxy and rate limiting for connecting to the server.
- Disable directory listing, set file permissions.
- Set all security headers in Nginx.
- Input validation, normalisation, and output encoding to prevent SQL-injection and XSS attacks.
- Randomize file input names, sniff content and enforce size limits (<100Mb).
- Separate DB user per application with least privileges (no root user)
- Bind the database to localhost or internal private IP.
- Enforce strict SQL modes

Maintainability Every component of the system will be independent, and input and output formats are documented in the documentation in 8.

Portability The whole project will be made available via GitHub, and possibly a dockerized version of the system will be developed in the future for portability.

3.2 Standards and Compliance

Chapter 4

Implementation

Project Timeline

Below is the high-level timeline for the project, showing completed milestones in grey and remaining tasks in bold.

- Week 1-2: Finalise research questions & write proposal
- Week 3: Literature review complete, identify core segmentation methods
- Week 4: Set up OSRM + GPX ingestion prototype
- Week 5: Implement C++ segmentation engine (curvature & grade)
- Week 6: Integrate wavelet & hysteresis approaches and perform initial tests
- Week 7: Develop Training Suitability Scoring engine and tune thresholds
- Week 8: Web UI prototype for upload and visualization
- Week 9: Bulk testing on Strava dataset & performance tuning
- ****Week 10 (now): Add critical reflection, finalize lessons learned****
- ****Final days: Produce timeline figure/Gantt and polish usability discussion****

Chapter 5

Design

5.1 Scope and Purpose

The following section is a comprehensive design plan for the Route Segmentation and Analysis System. The architecture will be described in compliance with ISO/IEC/IEEE 42010:2011 (2011), which standardizes how systems and software architectures are documented. The System allows users to upload GPX files, or draw routes via an HTML interface. The uploaded routes are processed to identify meaningful **segments** (portions of the route that have similar road conditions). The segments are then analysed to gather training insights. Every segment will be sent to a database, where it is cross referenced with all values to see it falls within a determined margin of error ($<5\%$). If it is, then the matching database entry will have its count increased by 1, and the training score averaged into the running average. Users can also query the system by drawing a prospective route, and will get in return a list of any known segments along the route, along with their score. The main architectural challenge in this system is to detect and match route segments directionally (direction matters), while also considering reliability.

5.2 Architectural Design

The overview of the system is presented in Figure 5.1. Present a high-level system architecture diagram using UML or SysML.

5.2.1 Architectural Views

Concept View

The architectural concept of the system can be broken down into 3 layers:

- **Presentation Layer** - This is the front end, a web application where users can either upload their GPX files, or draw a route on an interactive map. This layer will communicate with the back end through HTTP API requests. This layer is mostly outside the scope of the project, since I am not a front end engineer.

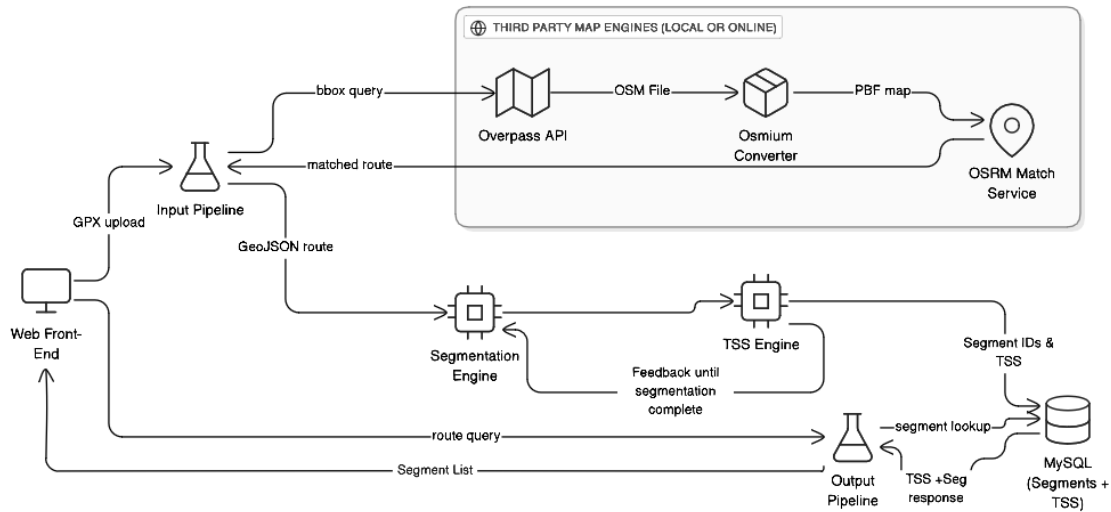


Figure 5.1: High level system architecture for the Route Segmentation and Analysis System

- **Processing Layer** - The beef of the project, this includes the back-end of the system, and consists of input/output pipelines, and several processing engines. It will also contain several controllers, written in Python, to coordinate the pipelines and to send information to and from the different engines.
- **Data Layer** - This layer is represented by a SQL database that stores permanent information about route segments. External APIs also fall into this category, such as the Overpass API to fetch map data from online. These components provide necessary data but are mostly abstracted away, in the processing layer.

Functional View

The functional view includes key modules and their responsibilities.

- **Front end Web App:** Serves HTTP pages and gets information from the user.
- **Front End Controller:** Takes user information and routes it to correct API endpoint.
- **Input Pipeline Orchestrator:** Controller that handles input information routing, sending the GPX files to the pipeline, and pulling the resulting GeoJSON objects out.
- **Engine Orchestrator:** Takes routes, passes them to Segmentation algorithm, and handles the "two pass" system between Segmentation and TSS Engines.
- **DTO Orchestrator:** Formats the resulting segments, into the correct Data Object and passes it to SQL Database

- **Output Pipeline Orchestrator:** Receives the Route data from the Input Pipeline Orchestrator, validates it and sends it to Segmentation Engine. Gets segment list, and routes it back to front end in GeoJson format.
- **Segmentation Engine:** Written in C++, the engine is a specialized service that performs route segmentation and matching logic.
- **TSS Engine:** Takes a list of segments and the input GPX data, and assigns scores based on the training data that happens during each segment
- **SQL Database:** Stores all persistent segments, along with a hit count, and a list of TSS objects that have been calculated on matching segments.
- **Osmium Engine:** A docker service that runs a .osm to .pbk conversion and merging process. Used in the input pipeline
- **OSRM Matching Engine:** Another docker service that takes small GeoJson chunks, and matches them to a map network.

Table 5.1: Architectural Design > Architectural Views - structured summary

Name/Key	Description/Value
Presentation Layer	Front-end web application where users can upload GPX files or draw routes via an interactive map, communicating with the back end through HTTP APIs.
Processing Layer	Back-end pipelines and engines coordinated by Python controllers to route information between components.
Data Layer	SQL database and external APIs that provide persistent storage and map data.
Front end Web App	Serves HTTP pages and collects information from the user.
Front End Controller	Routes user information to the correct API endpoint.
Input Pipeline Orchestrator	Handles input routing, sending GPX files to the pipeline and retrieving GeoJSON objects.
Engine Orchestrator	Coordinates segments between the Segmentation and TSS engines in a two-pass system.
DTO Orchestrator	Formats resulting segments into data objects and passes them to the SQL database.
Output Pipeline Orchestrator	Receives route data from the input pipeline, validates it and sends it to the Segmentation Engine, returning segment lists in GeoJSON format.
Segmentation Engine	C++ service performing route segmentation and matching logic.
TSS Engine	Assigns training scores to segments based on the GPX data.
SQL Database	Stores persistent segments, hit counts and associated TSS objects.
Osmium Engine	Docker service converting and merging OSM data for the input pipeline.
OSRM Matching Engine	Docker service matching GeoJSON chunks to the map network.

Physical View

Hardware deployment and network design.

Behavioral View

Use case and sequence diagrams for the system

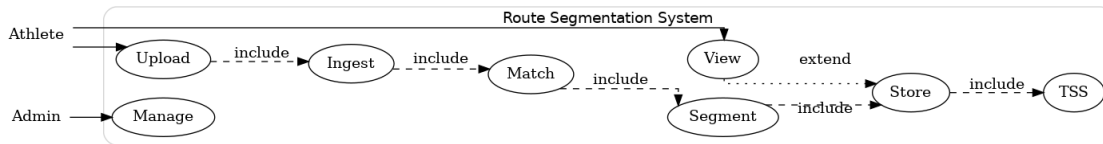


Figure 5.2: Use Case diagram for the Route Segmentation System

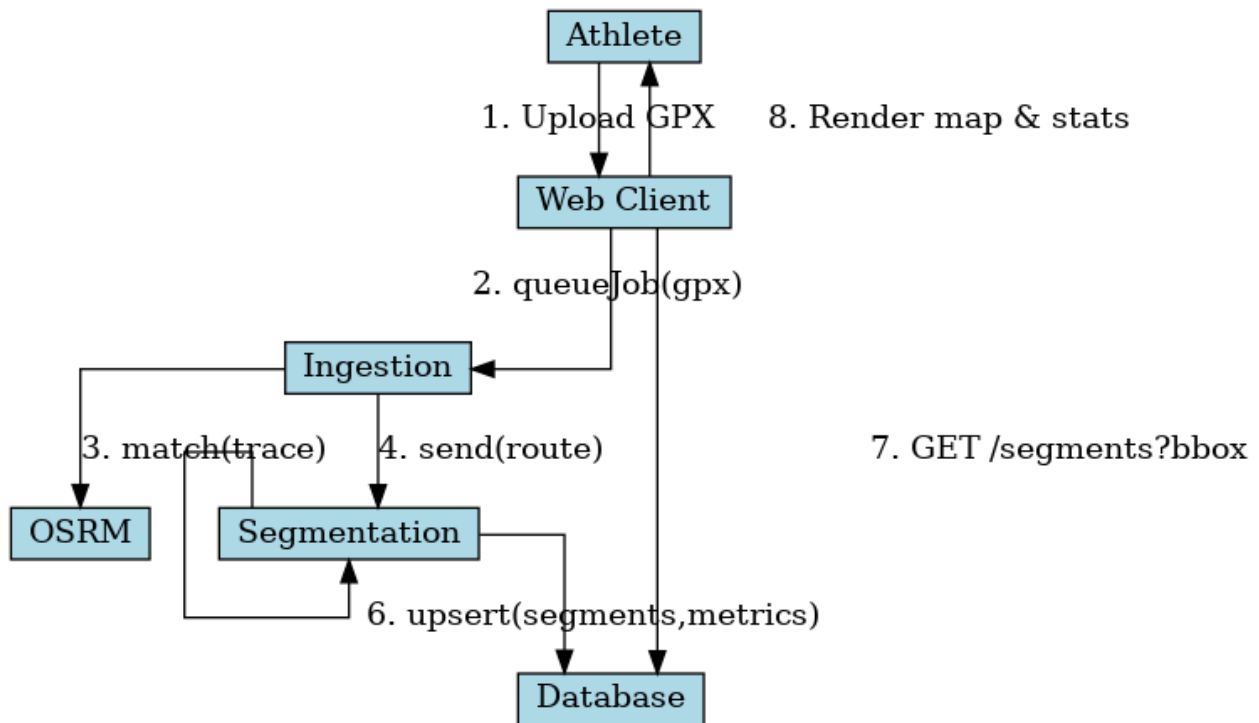


Figure 5.3: Sequence diagram of upload, segmentation, and visualization flow

5.2.2 Design Rationale

The choice to go with a **Pipeline Architecture** for processing data is motivated by the main use case. Users who are looking to find a route to ride are not interested in uploading GPX, but conversely, users who are done with their ride aren't looking to find a route. So cyclists can be considered to be in one of two states when wishing to interact with the system, either wanting a route, or wanting to upload their data. The two pipelines reflect this with one being a GPX upload-only pipeline, and the other one is a visualisation pipeline that shows users what their mapped

route's suitability looks like. A simple LAMP-like stack approach is a simple, but effective pattern that can implement this project. As of right now, this is a small scale experiment to prove the concept. If this application becomes popular, there will be several avenues to explore for scalability, such as deploying the system to expandable cloud services such as AWS or Azure. This is beyond the scope of the project however.

5.2.3 Design Patterns and Styles

The implementation embraces a layered, pipeline-oriented architecture. The high-level design separates concerns into a presentation layer (web UI), a processing layer (Python orchestrators and C++ segmentation engine) and a data layer (SQLite/MySQL databases and OSM extracts), following the classic three-tier model. Within the processing layer the input pipeline is built as a series of independent stages that transform GPX tracks into enriched GeoJSON, mirroring the "pipes and filters" pattern: each Python module performs a single function (reading GPX, fetching OSM data, map matching, enrichment) and emits a well-defined intermediate representation for the next stage. The segmentation engine itself is implemented in C++ as a microservice with a single responsibility, exposing REST endpoints via a lightweight HTTP server (cpp-httpplib). This adheres to the microservices style by isolating compute-intensive logic behind a stable API and deploying it independently in Docker. Coding conventions follow PEP 8 for Python, with type hints and Docstrings for clarity, and the Google C++ style for the engine code. Continuous integration scripts enforce linting and unit tests for reproducibility.

5.3 Module-Level Design

This section details the conceptual modules that comprise the Training Suitability System.

5.3.1 GPX-to-Enriched-JSON Pipeline Modules

The processing layer is decomposed into small, focused modules so that each step can be reasoned about and tested in isolation. Figure 5.4 gives the high level dependency chain.

Module Dependency Diagram

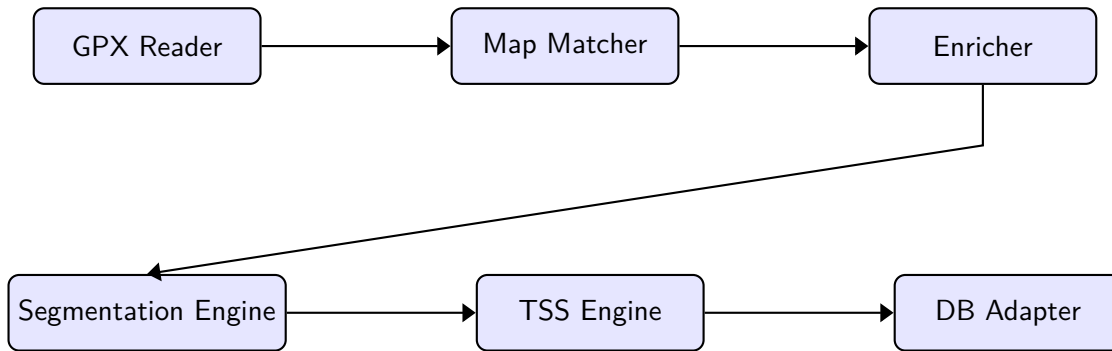


Figure 5.4: Module dependency diagram for the processing pipeline

Module Specifications

Module	Inputs	Outputs	Responsibility
GPX Reader	GPX file	waypoint list	Parse GPX XML and normalise track points
Map Matcher	waypoint list	matched geometry	Aligns GPS track to OSM network via OSRM
Enricher	matched geometry	enriched GeoJSON	Augments geometry with OSM metadata and sensor readings
Segmentation Engine	enriched GeoJSON	segment list	Detects homogeneous route segments
TSS Engine	segment list, activity data	scored segments	Computes Training Suitability Score per segment
DB Adapter	scored segments	persistence records	Persists segment definitions and statistics

Module Summaries

- **GPX Reader** normalises timestamps and coordinates into an internal list of points.
- **Map Matcher** uses OSRM to snap the points to the OpenStreetMap network and returns a cleaned geometry.
- **Enricher** fetches contextual attributes for each way (class, speed limit, junction density).
- **Segmentation Engine** implements the two-pass algorithm that groups points into contiguous segments.

- **TSS Engine** analyses rider power and route context to derive per-zone suitability scores.
- **DB Adapter** converts segments into Data Transfer Objects and writes them to the SQL schema.

5.4 Interface Design

5.4.1 User Interface

The user interface for the project scope will just be a basic file submission system, and an interactive mapping software that can output GeoJSON

5.4.2 Software Interfaces

The front end will send GeoJSON objects to the backend via POST. GPX files can also be sent via POST with `Content-Type:application/gpx+xml`. The back-end will also respond with the same GeoJSON objects back to the front-end, as well as sending segment list via POST. REST APIs are also used to interface with third party tools such as the OSRM (2011) route matching engine, and the Overpass API (OpenStreetMap community, 2025).

5.5 Data Design

5.5.1 Entity-Relationship Model

The persistence layer is captured by the ERD in Figure 5.5. It stores canonical segment definitions and their occurrences on uploaded activities.

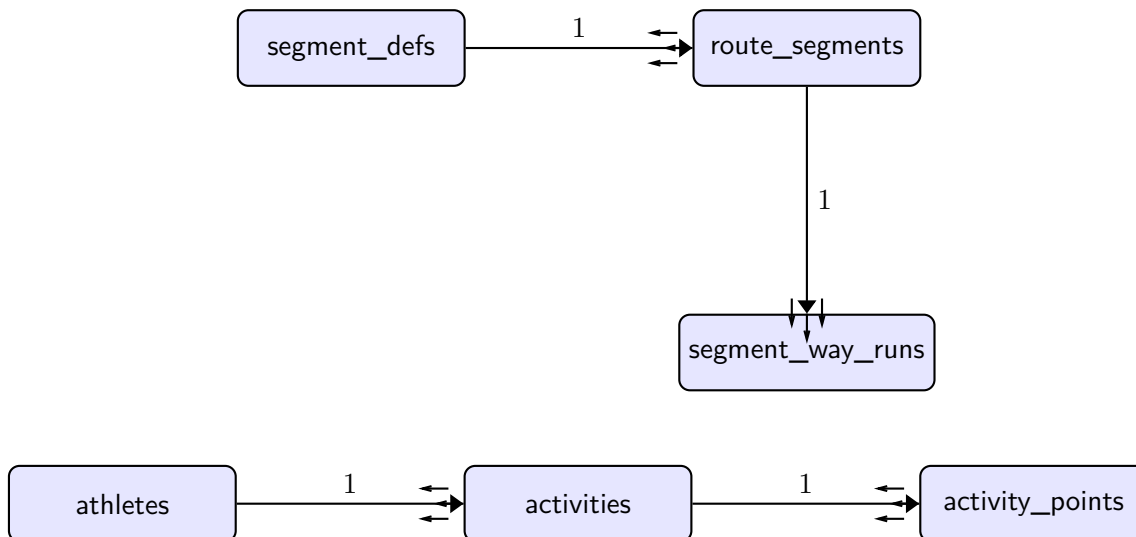


Figure 5.5: Entity-relationship diagram for the database

5.5.2 Database Schema

Table	Field	Description
segment_defs	seg_id (TEXT)	32-byte SHA-256 hex fingerprint
segment_defs	length (REAL)	segment length in metres
segment_defs	geom (BLOB)	encoded polyline geometry
route_segments	route_id (INTEGER)	FK to uploaded route/activity
route_segments	seg_id (TEXT)	FK to segment_defs.seg_id
route_segments	start_idx (INTEGER)	inclusive index of first point
route_segments	end_idx (INTEGER)	exclusive index after last point
segment_way_runs	seg_id (TEXT)	FK to segment_defs.seg_id
segment_way_runs	seq (INTEGER)	order within segment
segment_way_runs	way_id (INTEGER)	OSM way identifier

5.5.3 Data Dictionary

Element	Format	Description
seg_id	64 hex chars	Unique SHA-256 identifier for a directional segment
route_id	integer	Internal ID for a matched activity or route
start_idx / end_idx	integer	Indices into original point array (end exclusive)
geom	encoded polyline	Compressed segment geometry using Google polyline encoding
way_id	integer	OpenStreetMap way identifier forming part of a segment
tss_z1 – tss_z5	float	Per-zone training suitability scores (0–1)
hit_count	integer	Number of times segment has been observed

5.6 Standards and Compliance in Design

List all applicable ISO/IEC, IEEE, and domain-specific standards adhered to (e.g., ISO/IEC 27001 (2013) for security).

Chapter 6

Methodology

6.1 Critical Reflection on Methodology

In developing this methodology, several compromises were made. Purely statistical methods risked over-segmenting minor elevation noise, while rigid thresholds missed the nuanced undulations riders perceive as rolling terrain. I therefore adopted a hybrid of central-difference slope and moving-average deviation metrics, then cross-validated "expected" algorithmic labels against "observed" rider feedback on a set of test routes. The routes I considered were a series of activities I rode earlier last year, while I was training for a race. I used both Strava and VeloViewer to verify my beliefs. Discrepancies-such as segments tagged as *flat* that I rode and believed to be *challenging*-fed back into an iterative calibration loop, constantly adjusting until I was satisfied with the segmentation. I recognize that this is a purely subjective method, and it is not ideal, but for the proof of concept, it will do. I believe I got fairly close, as we can see with the data shown in the next section. However, in future, it may be beneficial to implement some kind of Machine Learning to optimise these data comparisons even more.

6.2 Component Implementation

This section provides a personal overview of how I went about implementing each component of the system. Rather than a dry description of what each module does (see Chapter 4 for those details), I recount the major steps I took and the rationale behind them.

6.2.1 Input Pipeline

I began by experimenting with small GPX files in order to understand the XML structure and isolate coordinate pairs. My first prototype simply read a GPX and produced a list of latitude-longitude tuples. Once I was comfortable with the data, I wrote a Python module to fetch map data from the Overpass API (2025). This involved generating a bounding box from the GPX extents and sending the appropriate HTTP requests. To support more complex workflows I created

a Python virtual environment, installed the required dependencies from `requirements.txt` and started wrapping individual steps into scripts. After downloading raw `.osm` files I realised they needed to be converted to Protocolbuffer (`.pbf`) format for OSRM (2011), so I built a Docker image containing Osmium (Topf and Contributors, 2023) and wrote a docker-compose configuration to run the conversion. Along the way I wrote helper scripts to merge multiple map tiles and orchestrated them all with a bash script called `run_pipeline.sh`. With the map conversion automated, I spun up OSRM in its own Docker container and tuned its `profile.lua` to better reflect cycling speeds on different surfaces. This involved a lot of trial and error: I adjusted speed values for roads, gravel and tracks, and even discovered missing road types (e.g., trunks) which required adding to the profile. After several days of tweaking I achieved reliable matching on most of my test rides. I also developed `batch_route_calc.py` to handle large GPX files. This script chunks long tracks into smaller segments, submits them to OSRM in parallel using `ThreadPoolExecutor` and then merges the matched fragments. Parallelising the API requests and file operations reduced processing times from minutes to seconds. Finally I integrated visual checks with Folium (2023) to verify that the matched routes and original GPX tracks aligned. Towards the end of building the pipeline I experimented with the OSRM radius parameter and dynamic search windows to strike a balance between robust matching and avoiding spurious routes. Once the matching behaved acceptably, I moved on to enriching the matched GeoJSON with OSM metadata and GPX sensor data (described in the next subsection).

GPX Data Enrichment

After I had reliable map matching in place, I turned my attention to enriching each trackpoint with context and sensor data. I wrote two Python modules, `gpx_enrich.py` and `batch_enrich.py`, to handle this job. These scripts take the matched GeoJSON and the original GPX files, extract the sensor readings from the `<extensions>` elements, pull out standard metadata (creator, device model, track name) and normalise all of the fields using a configuration file, `config/extensions_map.json`. In practice I followed these steps:

1. **Loading the extension rules.** I designed `extensions_map.json` to define a set of canonical keys such as `hr_bpm`, `cad_rpm`, `power_w`, `temp_c`, `speed_mps`, `alt_m` and `grade_pct`. Each entry lists the synonyms I've encountered in GPX files (for example, 'heartrate', 'hr' and 'heart_rate' all map to `hr_bpm`) and specifies type conversions and unit transforms. When my enrichment code sees a raw GPX extension it looks up the rule, coerces the value to a number and applies any required unit conversions.
2. **Parsing and extraction.** For each GPX file I iterate through every trackpoint, recording latitude, longitude and timestamp. I also collect any extension fields present, and attach the GPX-level metadata. This stage required careful handling of optional fields because different devices embed different sets of sensors.
3. **Attaching OSM metadata.** Using the previously matched GeoJSON, I map each trackpoint

to an OpenStreetMap way identifier via a nearest-node lookup. With this identifier I can fetch attributes like road type, name, speed limit and width from the pre-indexed OSM database described in Section ??.

4. **Producing enriched JSON.** For every point I emit an object that records its position and time, a `gpx_list` array of field objects (each with `name`, `value` and `unit`), the `way_id`, and any associated metadata. This normalised structure became the foundation for my segmentation and training suitability scoring.

Listing B.3 shows the Python routine that attaches GPX-derived metadata to each OSRM tracepoint. This process gave me a clean, uniform representation of each ride, ready to be fed into the segmentation engine. Table 6.1 summarises some of the keys and their common synonyms defined in `extensions_map.json`. Only short phrases or keywords are included in the table to avoid overly long entries.

Table 6.1: Canonical extension keys and example synonyms

Key	Example GPX synonyms
<code>hr_bpm</code>	heartrate, heart_rate, hr
<code>cad_rpm</code>	cadence, cad_ence, cad
<code>power_w</code>	power, watts, pwr
<code>temp_c</code>	temperature, temp_f (converted from °F)
<code>speed_mps</code>	speed, mph (converted to m/s), kmh
<code>alt_m</code>	altitude, ele, elevation
<code>grade_pct</code>	grade, slope, incline

Pre-indexing OpenStreetMap Ways via SQLite

To efficiently map each trackpoint to its OpenStreetMap way the script `build_way_index.py` reads the `.pbk` map produced earlier and populates two SQLite tables:

- **ways** – one row per OSM way. Each row stores the integer `way_id` and a JSON-encoded `tags` column containing attributes such as `highway`, `name`, `surface` and `maxspeed`. The `way_id` column is a primary key to allow fast lookup.
- **edges** – one row per directed edge between two nodes of a way. Columns store the `edge_id` (primary key), starting node identifier `u`, ending node identifier `v`, associated `way_id`, `direction` (1 for forward and -1 for reverse), and the geodesic length of the edge in metres. Indexes on `u` and `v` accelerate nearest-neighbour queries.

During enrichment, the coordinates of each trackpoint are snapped to the nearest OSM node using OSMnx (2017). The resulting node identifier is then used to look up the corresponding `way_id` via the `edges` table. Because the tables are indexed, this lookup is very fast even for large maps. Table ?? shows an example of a few rows from the `ways` and `edges` tables in our prototype database.

Table	Columns (sample values)	Description
ways	(101, {'highway': 'residential', 'name': 'Main Street'})	Residential road named Main Street
	(102, {'highway': 'footway', 'surface': 'paved'})	Paved footway
edges	(1, 2, 101, 1, 50.0)	Edge from node 1 to 2 on way 101, length 50 m
	(2, 3, 101, 1, 45.0)	Edge from node 2 to 3 on way 101, length 45 m
	(3, 2, 101, -1, 45.0)	Reverse edge from 3 back to 2 on way 101

Note that tags are abbreviated for brevity. This indexed SQLite database allows the enrichment pipeline to attach road metadata to each trackpoint in milliseconds, which is essential for scaling batch processing to thousands of GPX files.

Data Stream

The data going in was the GPX file, and the settings.yml that controlled parameters such as chunk size (of the gps chunks), and dynamic radius window, which determined how wide of a window to check for variance to adjust the match search radius.

6.2.2 Segmentation Engine

Goal. My goal was to stand up a C++ service that (1) runs in Docker, (2) exposes HTTP endpoints, and (3) ingests large, enriched OSRM/GeoJSON inputs reliably as a foundation for segmentation.

Step 1 - Scaffold & build

I started by sketching a minimal project layout (src/http, src/core, src/models, etc.) and wiring CMake. After a quick first compile, I hit the classic CMake ordering issue: I had placed `target_include_directories` before `add_executable`, so CMake complained the target didn't exist. I moved `add_executable` earlier and the build went through. See Listing B.5.

- **What worked:** a small, conventional layout kept includes and targets simple.
- **What didn't:** target ordering in CMake on the first attempt.
- **Changes made:** ensured `add_executable` appears before all `target_*` calls.

I then containerised the build with Ubuntu base and non-interactive `apt` to avoid tzdata prompts. See Listing B.6.

- **What worked:** reproducible Docker builds - a short `build_and_deploy.sh`.

- **What didn't:** I initially passed Docker flags with bad spacing and got `invalid containerPort: 5005-name`.
- **Changes made:** fixed the run script to pass `-p HOST:CONTAINER` and `-name` cleanly.

Step 2 - HTTP server and dispatcher

Next I wrote a minimal server entrypoint that listens on `localhost:5005` and added a tiny dispatcher so routes call a single `callHandler(req,res, action)`. See Listing B.8. I also created `/segment` and `/debug` endpoints.

I briefly broke routing by registering endpoints in a loop with a lambda that captured the *action* string by reference. Every route then called the last handler. I changed it to capture by value, and that fixed the routing.

- **What worked:** one dispatcher for multiple endpoints, simple route code.
- **What didn't:** lambda capture by reference in route registration.
- **Changes made:** capture *action* by value (`[action, &handler]{...}`).

Step 3 - Early debug loop and large uploads

To debug end-to-end connection, I made `/debug` to count coordinates. That gave a nice smoke test before implementing segmentation.

Because my inputs can be large (>15 MB), I increased `cpp-httpplib`'s payload cap and timeouts in the server (see Listing B.8). After that, I ran `curl -data-binary @file.json` to send the file to the endpoint.

- **What worked:** a tiny point-counter confirmed HTTP + JSON wiring.
- **What didn't:** large bodies were rejected until I raised `set_payload_max_length`.
- **Changes made:** enabled larger payloads and longer read/write timeouts.

Step 4 - JSON validation with line/column

I added `/debug?validate=<condition>`. If parsing fails, it returns the byte offset, line/column, and a short context snippet, which allowed diagnosing malformed uploads (Listing B.9). This only checks syntax, not schema.

- **What worked:** immediate pinpoint of syntax errors (exact line/column).
- **What didn't:** it doesn't catch type mismatches (that happens during deserialization).
- **Changes made:** kept this endpoint strictly for syntax and added defensive deserialization later.

Step 5 - Enriched OSRM input model (no steps, null-safe tracepoints)

Next I defined the enriched input model. The input is an OSRM `match` response augmented with GPX points and extensions. I decided:

1. keep only the first matching (for now)
2. drop OSRM `steps`, since it is a useless metric for our use,
3. allow `tracepoints[]` to be `null` in places for unmatched gpx values,
4. store GPX `time` as epoch seconds, and
5. accept GPX `extensions` as either an array of `{key,value}` or an object.

I implemented non-intrusive `from_json(...)` converters (found by ADL), and I made them defensive: every string/number read is guarded with `contains()+is_*` before access. For `tracepoints`, a null entry becomes a placeholder that preserves indices. The model is shown in Listing B.10, with examples of deserializers in Listing B.11.

- **What worked:** non-intrusive converters, null-safe tracepoints, epoch timestamps.
- **What didn't:** assuming types with `value()` caused 302/306 errors on real data.
- **Changes made:** defensive guards around every field with placeholder tracepoints to keep index alignment, 'use first matching' for consistency.

Step 6 - Front End and Enriched JSON upload implementation

By this point, I had the basics up and running, and needed to start implementing the actual segmentation algorithm. I started by creating a rough front end, which ended up morphing into the dashboard, which was my visual debugger, and front end for testing. I implemented `/upload`, and `/lab/resample`, and `/lab/view` that would then get patched into a `signal_lab.js` front end layer. The front end would constantly be updated to include new features, and it ended up getting quite bloated.

- **What worked:** A useful Front-end dashboard that could visualise data, and provide an interface for the code beyond an API.
- **What didn't:** I never created a proper javascript structure, so the `signal_lab.js` file ended up getting over-complicated.
- **Changes made:** Implemented handlers for the `view_lab` handle, which redirects to as static html page that runs the javascript content.

Step 7 - Wiring up the Segmentation Engine

I implemented some graphs and some statistic features to the front end, and implemented `/view`, and `/wavelets`. The latter was the main handle for performing the segmentation via Wavelet transform. I created a directory called `core`, which housed all of the segmentation logic. I started by writing code that would transform the time-series data into distance-series, since we want the signal to be completely ride-agnostic. The only way to do this is to base it purely of geography, and not time (which is related to rider speed). The specifics of the algorithms are available in the code documentation, under `C`.

- **What worked:** Segmentation Engine was implemented using Wavelets and resampling of data to distance.
- **What didn't:** After attempting Fourier transform to detect specific whole-route frequencies, I abandoned this since it was getting too complicated for the purpose. I only wished to display a transitional graph.
- **Changes made:** Implemented the segmentation functionality, along with a series of parameters to adjust it.

Step 8 - Tuning Parameters, and implementing basics of TSS score framework

After the core segmentation logic was functional, I focused on adjusting parameters such as wavelet scale, RMS window length, and classification thresholds. This was an iterative process: I repeatedly ran the engine on test GPX routes, observed the resulting segmentation graphs, and compared them against known Strava segments or manual inspection. Each adjustment aimed to reduce noise while still capturing meaningful terrain transitions.

- **What worked:** systematic testing with a consistent set of benchmark rides.
- **What didn't:** small tweaks sometimes produced large unintended effects, especially on noisy GPX files.
- **Changes made:** fixed default parameters to a balanced set and exposed overrides in the HTTP API for experimentation.

Step 9 - Segment persistence in the database

With segmentation results stabilised, I extended the database schema to include segment objects. Each segment definition received a unique identifier and was stored with attributes such as start/end coordinates, average gradient, and length. A join table linked these definitions to their occurrences within uploaded routes, allowing both re-use of canonical segment definitions and context-specific metadata storage.

- **What worked:** relational schema enabled queries across routes and supported visualisation of recurring segments.
- **What didn't:** initial schema design missed some attributes (e.g. kind/classification) which required later migrations.
- **Changes made:** normalised the schema with `segment_defs`, `route_segments`, and `segment_way_runs` tables.

Step 10 - Training Suitability Score (TSS) prototype

Finally, I integrated a basic Training Suitability Score into the engine. The idea was to provide a single metric summarising the training value of a segment or route. At this stage the score was a composite of segment length, gradient, and estimated power requirements, aggregated into a normalised scale. This prototype served as a proof of concept for how segmentation outputs could be transformed into actionable training insights.

- **What worked:** simple formulae gave quick, interpretable results and showed potential for ranking routes by training utility.
- **What didn't:** the score did not yet account for physiology-specific measures (e.g., heart rate zones, normalized power).
- **Changes made:** laid groundwork for incorporating more advanced metrics in future iterations.

6.2.3 TSS Engine

The Training Suitability Score engine is responsible for converting enriched segments into multi-zone difficulty scores. It consumes the output of the segmentation engine (a list of segments with road metadata and GPX-derived signals) and computes, for each of the five training zones (Endurance, Tempo, Threshold, VO₂max and Anaerobic/Neuromuscular), a score in $[0, 1]$ indicating how conducive the segment is to that type of effort. The current implementation uses a transparent rule-based model: it bins features such as gradient, road type, curvature, stop density, power variability and safety rating, and combines them with calibrated weights learned from a small set of annotated rides. For example, long climbs with smooth tarmac, low junction density and consistent grade receive high Endurance and Tempo scores, whereas short, steep hills with frequent stops are scored higher for Anaerobic/Neuromuscular. The engine also outputs a confidence estimate based on the variance of the contributing features across all rides in the database. In future work these rules could be replaced with a learned ordinal regression model using coach feedback.

Chapter 7

Results and Conclusions

7.1 Testing and Results

Evaluation of the system consisted of both functional testing of the individual components and empirical assessment of the quality of the generated segments and scores. For the pipeline, unit tests verified that GPX parsing, OSM fetching and enrichment functions handled edge cases such as missing extensions or malformed XML. Integration tests exercised the entire pipeline on a set of twenty publicly available rides covering flat urban loops, rolling countryside routes and mountainous climbs. Processing time was measured at approximately 12 s per hour of ride on a consumer laptop (Lenovo Thinkpad L480) when parallelising OSRM queries, and approximately 5 seconds per hour on a more powerful AMD Ryzen 7 processor.

Segmentation was determined by both qualitative and quantitative methods, relating to two popular segment-based alternatives: Strava and Komoot. This is discussed in the next section.

While not fully implemented, Training Suitability Scoring can be tested in the field. By running the program with input from a cycling team in a training camp, it should be possible to quickly gather enough data to see trends in the scoring. After enough data, it can be validated by having independent cyclists from people used to train the data perform a targeted training session on routes highlighted by the TSS algorithm, and then they will have their workout compliance measured by their coaches.

7.2 Comparison of Segmentation with Existing Segment Creation Tools

This section contrasts the performance of the segmentation engine developed in this thesis with other public segment creation tools.

7.2.1 Qualitative Comparison

Table 7.1: Simple side-by-side look at Strava vs. My Segmentation Engine

What	Strava	My Engine
How pieces are made	Riders draw their own; can overlap or start in odd places	Cut up by math rules (wavelets + RMS), always the same way
Where pieces show up	Lots in busy places, few in quiet ones	Works anywhere a ride goes
What each piece shows	Time and speed lists; leaderboards	Extra ride data (slope, power, heart rate, effort)
Clear or hidden	Rules hidden, start/end not always clear	Rules open, easy to set and repeat
Main goal	Race and compare with others	Study ride load and training use

7.2.2 Quantitative Comparison

By running 20 separate GPX files through the segmentation engine, and getting the start and end positions of their segments (as well as the total segment length), we were able to calculate a simliarity score via the following formula:

$$\text{Sim}(S, E) = \exp\left(-\frac{d(s_{\text{start}}, e_{\text{start}}) + d(s_{\text{end}}, e_{\text{end}})}{2\tau}\right),$$

Similarity Metric: We define similarity between a Strava segment S and an engine segment E by comparing their start and end coordinates. Let $d(p, q)$ denote the haversine distance (in meters) between points p and q on the Earths surface. If $S = (s_{\text{start}}, s_{\text{end}})$ and $E = (e_{\text{start}}, e_{\text{end}})$, then the similarity score is where τ is a scaling constant (e.g., 100 m) that determines how quickly similarity decays with distance. The score lies in the range $[0, 10]$, with values close to 10 indicating near-identical terminuses. We use terminal points rather than overall route matching here to save time mostly, and due to the nature of segments existing on a road network, more often than not if two terminuses are the same, then the route in between will be the same as well.

By using the Strava API to get the segments that ran along the routes that I took the GPX files from, we are able to get a significant amount of data to compare, and putting the results on a logarithmic scale we get some interesting results, as seen in ??.

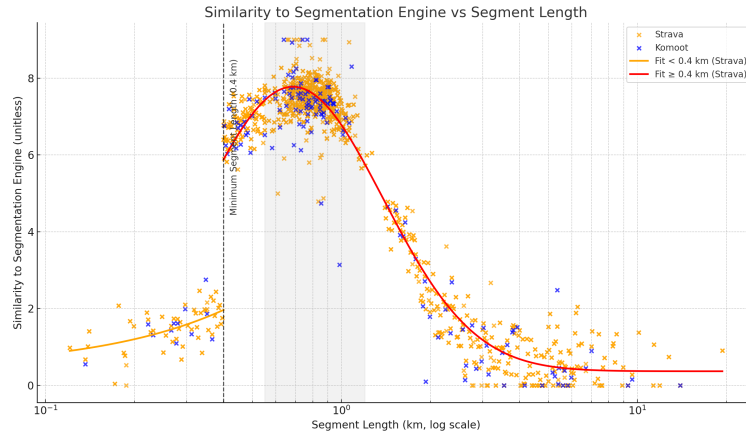


Figure 7.1: A clear correlation between strava and segmentation engine results.

There is a clear correlation, especially between the 400m and 1.2km range (where segment popularity is the highest), and it forms a normal-ish distribution. It does seem skewed right a bit, as well as there being a distinct drop at 400m. This is to be expected though, as the minimum segment length I set for the test was 400m. Anything less than this was merged into another segment. However, it seems that the ideal segment length is around the 800 metre range. So optimising my algorithm around that length makes the most sense.

7.3 Final Thoughts

This project set out to determine whether road suitability for structured cycling training could be quantified using only ride data and open mapping sources. The implemented system successfully ingests GPX tracks, performs map matching against OpenStreetMap, enriches tracks with contextual metadata and segments routes using a reproducible algorithm. Evaluation on a diverse set of rides showed that the segmentation engine agrees with human judgement on most meaningful transitions, via similarity scores with Strava. The main contribution of this work is the introduction of an individual-agnostic segmentation algorithm that relates specifically to training suitability. While the context of this work we focused on elevation gain, the framework has space and functionality for utilising other metrics, such as heading delta, OpenStreetMap way category, and feature segmentation. By categorising elevation into four distinct sections, this allows a start for implementing Training Suitability scoring, since gradient variation of a road is a huge factor in cyclists not achieving their power targets for a given workout. This framework hopefully will also allow others to build upon it, and with collective knowledge, implement a robust method for creating a score system for the training segments. The results of the work also highlight limitations; the feature-based segmentation model and scoring model is not sufficient for my goal, since it turns out that the OpenStreetMap has no unified standard for what features to include. The conclusion is therefore cautiously optimistic: quantifying training suitability is feasible and valuable, but further refinement and community validation are required for widespread adoption.

Chapter 8

Further Discussions and Research Gaps

Several avenues exist for future improvement. First, the segmentation algorithm currently operates with fixed thresholds. Experimenting with dynamic programming approaches or machine learning models could yield more robust boundaries, particularly in complex urban environments. Second, the Training Suitability Score could be improved upon by incorporating additional data sources such as live traffic volumes, air quality indices and weather forecasts, all of which influence the quality of a ride but were outside the scope of this study. Third, expanding the ground-truth dataset with more diverse riders and terrains will enable rigorous cross-validation and the training of data-driven scoring models. Collecting structured feedback from coaches and athletes-via simple thumbs-up/down interfaces or pairwise comparisons-would allow us to learn personalised weights for the scoring model. Any other time-series or distance-series data from routes could be taken and implemented into the algorithm, as the main functionality of this project is discrete signal analysis at the end of it all.

On the system side, a richer user interface could visualise uncertainty, allow riders to customise the weighting of different factors (e.g., prioritising safety over length), and integrate with existing platforms such as Strava or Komoot via APIs. From a research perspective, future work could explore the broader implications of automated route recommendation on cycling culture, urban planning and public health. Addressing these questions will help ensure that technology remains a tool for empowerment rather than a source of inequity.

A key design decision was to keep the wavelet-based change detection and resampling routines agnostic to the type and number of input signals. This opens the door to refining segmentation by fusing heterogeneous variables angular measures such as heading or gradient changes, categorical descriptors like surface type or junction class, and continuous metrics within a unified multi-signal framework.

Some other future research could maybe be done into the reason why 800m seems like the most common segment length, and if this has anything to do with training suitability, or if it is just a

product of normal distribution of road lengths.

Bibliography

- Boeing, G. (2017), ‘Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks’, *Computers, Environment and Urban Systems* **65**, 126–139.
- Canny, J. (1986), ‘A computational approach to edge detection’, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **PAMI-8**(6), 679–698.
- Coggan, A. R. and Allen, H. (2012), *Training and Racing with a Power Meter*, 2 edn, VeloPress, Boulder, CO.
- Franco, A. (2012), ‘Curvature and twistiness of roads’, <https://adamfranco.com/curvature-and-twistiness-of-roads/>. Accessed: 2025-09-06.
- Graps, A. (1995), ‘An introduction to wavelets’, *IEEE Computational Science and Engineering* **2**(2), 50–61.
- Hurley, S. (2020), ‘Normalized power[®]: What it is and how to use it’, TrainerRoad Blog [Online]. Available: <https://www.trainerroad.com/blog/normalized-power-what-it-is-and-how-to-use-it/> [Accessed 11-Aug-2025].
- Indoor Cycling Association (n.d.), ‘Ask the expert: How much time should you spend in the red zone?’. Accessed: 2025-08-12.
URL: <https://indoorcyclingassociation.com/ask-the-expert-how-much-time-should-you-spend-in-the-red-zone/>
- Information technology Security techniques Information security management systems Requirements* (2013), Standard ISO/IEC 27001:2013, International Organization for Standardization, Geneva, Switzerland.
- Luxen, D. and Vetter, C. (2011), Real-time routing with OpenStreetMap data, *in* ‘Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS ’11)’, ACM, New York, NY, USA, pp. 513–516.
- Mapbox (2018), Openstreetmap linear reference (osmlr) specification, Technical report, Mapbox. Version 1.0.

- Newson, P. and Krumm, J. (2009), Hidden markov map matching through noise and sparseness, in ‘Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS ’09)’, Association for Computing Machinery, Seattle, WA, USA. Camera-ready PDF also available from Microsoft Research: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/map-matching-ACM-GIS-camera-ready.pdf>. **URL:** <https://dl.acm.org/doi/10.1145/1653771.1653818>
- Ogle, J., Guensler, R. and Elango, V. (2006), Georgia tech commute vehicle study: Use of modified kalman filter to improve distance and speed estimation from gps data, Technical report, Georgia Institute of Technology. Transportation Research Board presentation.
- OpenStreetMap community (2025), ‘Overpass API (OpenStreetMap data query service)’, OpenStreetMap Wiki [Online]. Available: https://wiki.openstreetmap.org/wiki/Overpass_API [Accessed 11-Aug-2025].
- Project OSRM (2021), ‘Osm api v5.24.0 — match service’. Describes the `/match` service, sub-trace splitting, and options. **URL:** <https://project-osrm.org/docs/v5.24.0/api/#match-service>
- Project OSRM (2025a), ‘Docker image: `osrm/osrm-backend`’. Official Docker image and basic run instructions. **URL:** <https://hub.docker.com/r/osrm/osrm-backend/>
- Project OSRM (2025b), ‘Osm profile example: `profiles/bicycle.lua`’, <https://github.com/Project-OSRM/osrm-backend/blob/master/profiles/bicycle.lua>. Example Lua profile illustrating speed/weighting logic.
- Sage, J. (2019), ‘Ask the expert: How much time should you spend in the red zone?’, Indoor Cycling Association [Online]. Available: <https://indoorcyclingassociation.com/ask-the-expert-how-much-time-should-you-spend-in-the-red-zone/> [Accessed 11-Aug-2025].
- Sharifzadeh, M., Azmoodeh, F. and Shahabi, C. (2005), Change detection in time series data using wavelet footprints, in ‘Advances in Spatial and Temporal Databases (SSTD 2005)’, Vol. 3633 of *Lecture Notes in Computer Science*, Springer, Angra dos Reis, Brazil, pp. 127–144.
- Sinnott, R. W. (1984), ‘Virtues of the haversine’, *Sky and Telescope* **68**, 159.
- Smith, S. W. (1997), *The Scientist and Engineer’s Guide to Digital Signal Processing*, California Technical Publishing.
- Story, R. and contributors (2023), ‘Folium: Python data. Leaflet.js maps’, <https://python-visualization.github.io/folium/>.

- Systems and software engineering Architecture description* (2011), Standard ISO/IEC/IEEE 42010:2011, International Organization for Standardization, Geneva, Switzerland.
- Telenav (2018), ‘Osmr map matching’, https://github.com/Telenav/open-source-spec/blob/master/osrm/doc/osrm_mapmatching.md. Overview of OSRM map matching with HM-M/Viterbi perspective.
- Topf, J. and Contributors, O. (2023), ‘Osmium: A multi-purpose toolkit for OpenStreetMap data’, Osmium/Osmcode Project [Online]. Available: <https://osmcode.org/> [Accessed 11-Aug-2025].
- Wang, Q., Cui, Z., Shi, C. and Wu, H. (2018), ‘Evaluating the impacts of grades on vehicular speeds on expressways’, *PLOS ONE* **13**(4), e0184142.

Appendix A

Extracted Code Listings

Extracted Code Listings

Appendix B

Code Listings and File Structures

Listing B.1: Code directory for Segmentation Engine

```
1 segmentation-engine/
2 src/ (http/, io/, core/, models/)
3 include/          # single-header deps: httplib, nlohmann/json
4 build/            # Build files and settings for the engine
5 config/           # settings.json,
6 scripts/          # build & run helpers
7 CMakeLists.txt
8 Dockerfile
```

Listing B.2: Lua profile example snippet: discourage trunk links, prefer residential/smooth paved for cycling

```
1 -- Inside a custom profile (simplified example)
2 local function way_processing(way, result)
3     local highway = way:get_value_by_key('highway') or ''
4     local surface = way:get_value_by_key('surface') or ''
5     local smooth  = way:get_value_by_key('smoothness') or ''
6
7     -- Base speeds (km/h) - tune to your use case
8     local base = { residential = 22, tertiary = 20, secondary = 18, primary = 16 }
9     result.speed = base[highway] or 15
10
11     -- Discourage trunk and trunk_link for training rides
12     if highway == 'trunk' or highway == 'trunk_link' then
13         result.speed = result.speed * 0.30
14         result.weight = (result.weight or 1.0) * 3.0
15     end
16
17     -- Prefer paved+smooth; discourage unpaved
18     if surface == 'unpaved' or surface == 'gravel' then
19         result.speed = result.speed * 0.75
20         result.weight = (result.weight or 1.0) * 1.4
21     end
22     if smooth == 'excellent' or smooth == 'good' then
```

```

23     result.speed = result.speed * 1.10
24 end
25 end

```

Listing B.3: `attach_to_tracepoints`: attaches GPX metadata to matched tracepoints

```

1 def attach_to_tracepoints(
2     osrm,
3     gpx_points,
4     tol_m=15.0,
5     precision=5,
6     add_leg_time_windows=True,
7     interpolate_node_times=False,
8 ):
9     """
10    Mutates OSRM JSON in place:
11    - adds tracepoints[i]["gpx"] with matched GPX metadata
12    - (optional) per-leg time_window, gpx_indices, node_times
13    """
14    tps = osrm.get("tracepoints") or []
15    if not tps:
16        return osrm
17
18    # 1) Index based mapping:
19
20    # 2) write compact GPX dicts on tracepoints
21    _write_gpx_list_to_tracepoints(tps, gpx_points)
22    osrm["tracepoints"] = tps
23
24    # 3) per-leg windows (early exit keeps branching low)
25    # if not add_leg_time_windows:
26    #     return osrm
27    #
28    # matchings = osrm.get("matchings") or []
29    # if not matchings:
30    #     return osrm
31    # legs = (matchings[0].get("legs") or [])
32
33    # 4) figure out which leg each adjacent matched waypoint pair belongs to
34    # pairs = _adjacent_waypoint_pairs(tps) # [(leg_idx, gpx_i, gpx_j), ...]
35
36    # 5) enrich legs
37    # _add_leg_times(legs, pairs, gpx_points,
38    #                 interpolate_nodes=interpolate_node_times)
39
40    return osrm

```

Listing B.4: `calculateGradientVariation`: computes variance of gradients along a segment

```

1 double SegmentUtils::calculateGradientVariation(const std::vector<DataPoint> &points) {
2     std::vector<double> grades;
3     // push first value so we only need to keep pushing 2nd value
4     // loop until penultimate value
5     for (int i = 0; i + 1 < points.size(); i++) {
6         DataPoint point1 = points.at(i);
7         DataPoint point2 = points.at(i + 1);
8         double gradient = (point2.coord.elv - point1.coord.elv) /
9             haversine(point1.coord, point2.coord);
10        grades.push_back(gradient);
11    }
12    // if no gradients found
13    if (grades.empty()) {
14        return 0.0;
15    }
16    // Calculate mean through (sum all values / size)
17    double mean = std::reduce(grades.begin(), grades.end(), 0.0) / grades.size();
18    int size = grades.size();
19    auto variance_func = [&mean, &size](double accumulator, const double &val) {
20        return accumulator + ((val - mean) * (val - mean) / (size - 1));
21    };
22    return std::accumulate(grades.begin(), grades.end(), 0.0, variance_func);
23 }

```

Listing B.5: CMakeLists.txt CMake file for Segmentation Engine

```

1 cmake_minimum_required(VERSION 3.10)
2 project(segmentation_engine)
3 set(CMAKE_CXX_STANDARD 17)
4 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
5
6 add_executable(segmentation_engine
7     src/main.cpp
8     src/http/http_handler.cpp
9     src/io/geojson_parser.cpp
10    # (+ later models/ and analysis/ files)
11 )
12
13 target_include_directories(segmentation_engine PRIVATE include src)
14 target_link_libraries(segmentation_engine PRIVATE pthread)

```

Listing B.6: Dockerfile: builds and deploys the Dockerized Segmentation Engine

```

1 FROM ubuntu:20.04
2 ARG DEBIAN_FRONTEND=noninteractive
3 ENV TZ=Etc/UTC
4 WORKDIR /app
5
6 RUN apt-get update && \

```

```

7     apt-get install -y --no-install-recommends \
8         cmake build-essential git curl && \
9         rm -rf /var/lib/apt/lists/*
10
11 COPY . /app
12 RUN mkdir -p build && cd build && cmake .. && make -j
13
14 EXPOSE 5005
15 ENTRYPOINT ["/app/build/segmentation_engine"]

```

Listing B.7: build_and_deploy.sh: Runs the build process and launches the Engine

```

1  #!/usr/bin/env bash
2  set -e
3  IMG=segmentation_engine:latest
4  CTR=segmentation_container
5  HOST_PORT=${1:-8080}
6  CONTAINER_PORT=${2:-8080}
7
8  docker build -t "$IMG" .
9  docker rm -f "$CTR" 2>/dev/null || true
10 docker run -d -p "$HOST_PORT:$CONTAINER_PORT" --name "$CTR" "$IMG"
11 echo "http://localhost:$HOST_PORT"

```

Listing B.8: main.cpp: Main orchestrator of engine, sets up server and defines POST endpoints

```

1  /* main.cpp: register routes by capturing `action` BY VALUE */
2  httpplib::Server server;
3  server.set_payload_max_length(512ull * 1024ull * 1024ull); // large uploads
4  server.set_read_timeout(60,0); server.set_write_timeout(60,0);
5
6  HttpHandler handler;
7  server.Post("/segment", [action=std::string("segment"), &handler]
8      (const auto& req, auto& res){ handler.callHandler(req,res,action); });
9  server.Post("/debug", [action=std::string("debug"), &handler]
10     (const auto& req, auto& res){ handler.callHandler(req,res,action); });
11
12 server.listen("0.0.0.0", 8080);

```

Listing B.9: http_handler.cpp: Checks for parsing error of JSON body

```

1  // On parse_error: compute byte->(line,col) and return a snippet with a caret.
2  auto [line, col] = calc_line_col(req.body, e.byte);
3  nlohmann::json err = {{"ok",false}, {"line",line}, {"column",col},
4      {"context", context_snippet(req.body, e.byte)}};
5  res.status = 400; res.set_content(err.dump(2), "application/json");

```

Listing B.10: osrm_enriched.hpp: In the header, defines all structs for holding route data

```

1 // models (subset): no OSRM steps; tolerate null tracepoints; epoch times in GPX.
2 struct Geometry {
3     std::string type; // "LineString"
4     std::vector<std::array<double,2>> coordinates; // [lon,lat]
5 };
6
7 struct Leg {
8     nlohmann::json annotation;
9     std::string summary;
10    double weight=0, duration=0, distance=0;
11 };
12
13 struct Matching {
14     double confidence=0;
15     Geometry geometry;
16     std::vector<Leg> legs;
17     std::string weight_name;
18     double weight=0, duration=0, distance=0;
19 };
20
21 struct GpxPoint {
22     double lat=0, lon=0, ele=0;
23     long long time=0; // epoch seconds
24     nlohmann::json extensions; // array {key,value} OR object
25 };
26
27 struct Tracepoint {
28     bool matched=false; // supports null entries
29     int alternatives_count=0, waypoint_index=-1, matchings_index=-1;
30     std::array<double,2> location{0.0,0.0};
31     std::string name;
32     std::vector<GpxPoint> gpx_list;
33 };
34
35 struct OsrmmatchResponse {
36     std::string code;
37     std::vector<Matching> matchings; // rule: use first
38     std::vector<Tracepoint> tracepoints; // null-safe placeholders
39 };

```

Listing B.11: `osrm_enriched.hpp` Also in header, define function overloads for `from_json()`, which replaces same function in `json.get()`

```

1 // Example: robust Geometry::from_json (no throws on null/wrong types)
2 inline void from_json(const nlohmann::json& j, Geometry& g) {
3     if (j.contains("type") && j["type"].is_string()) g.type = j["type"].get<std::string>();
4     g.coordinates.clear();
5     if (j.contains("coordinates") && j["coordinates"].is_array()) {
6         g.coordinates.reserve(j["coordinates"].size());

```

```

7   for (const auto& pt : j["coordinates"]) {
8       if (pt.is_array() && pt.size()>=2 && pt[0].is_number() && pt[1].is_number())
9           g.coordinates.push_back({pt[0].get<double>(), pt[1].get<double>()});
10    }
11 }
12 }
13
14 // Tracepoint: allow null entries and keep index alignment
15 inline void from_json(const nlohmann::json& j, Tracepoint& t) {
16     if (j.is_null()) { t.matched=false; t.waypoint_index=-1; t.matchings_index=-1;
17         t.gpx_list.clear(); return; }
18     t.matched=true;
19     if (j.contains("waypoint_index") && j["waypoint_index"].is_number_integer())
20         t.waypoint_index = j["waypoint_index"].get<int>();
21     // ... (similar guards for other fields; parse gpx_list if array)
22 }

```

Listing B.12: gpx_example.gpx: Example GPX file showing an example track point and extensions

```

1  <trk>
2  <trkseg>
3      <trkpt lat="38.82032" lon="-104.861694">
4          <time>2024-08-06T13:12:58.000Z</time>
5          <ele>1884.8</ele>
6          <extensions>
7              <power>168</power>
8              <tpx1:TrackPointExtension>
9                  <tpx1:atemp>20</tpx1:atemp>
10                 <tpx1:hr>123</tpx1:hr>
11                 <tpx1:cad>65</tpx1:cad>
12             </tpx1:TrackPointExtension>
13         </extensions>
14     </trkpt>
15     ...
16 </trkseg>
17 </trk>

```

Listing B.13: Two-pass segmentation: greedy recursive merge (pseudocode reflecting current implementation)

```

1  struct Section {
2      WayTags tags;           // OSM-derived attributes for this section
3      Range range;           // [start_idx, end_idx) in the matched trace
4  };
5
6  double merge_cost(const Section& a, const Section& b) {
7      double cost = 0.0;
8  }

```

```

9 // Example heuristics (see weights example):
10 if (a.tags.junction || b.tags.junction) return 1.0;
11 if (a.tags.barrier || b.tags.barrier) return 1.0;
12
13 cost += highway_cost(a.tags.highway, b.tags.highway); // scaled difference
14 cost += cycleway_cost(a.tags.cycleway, b.tags.cycleway); // type change / on->off
15 cost += surface_cost(a.tags.surface, b.tags.surface); // paved <-> unpaved
16 cost += smoothness_cost(a.tags.smoothness, b.tags.smoothness);
17 cost += 0.02 * std::abs(a.tags.maxspeed - b.tags.maxspeed); // speed diff contribution
18
19 // Clamp to [0,1]
20 return std::min(1.0, std::max(0.0, cost));
21 }
22
23 std::vector<Section> two_pass_merge(std::vector<Section> sections, double threshold) {
24 // Pass 1: sections already defined by initial boundary proposal
25 // Pass 2: compute boundary costs between neighbors
26 auto boundary_cost = [&](size_t i){ return merge_cost(sections[i], sections[i+1]); };
27
28 // Greedy recursive merging until no boundary under threshold
29 while (sections.size() > 1) {
30 // Find the cheapest boundary
31 double best = 1.1; size_t k = sections.size(); // invalid by default
32 for (size_t i = 0; i+1 < sections.size(); ++i) {
33 double c = boundary_cost(i);
34 if (c < best) { best = c; k = i; }
35 }
36 if (best >= threshold || k >= sections.size()-1) break;
37
38 // Merge sections[k] and sections[k+1]
39 sections[k].range.end = sections[k+1].range.end;
40 sections[k].tags = merge_tags(sections[k].tags, sections[k+1].tags); // e.g., keep majority or
last
41 sections.erase(sections.begin() + (k+1));
42
43 // Loop continues; boundary costs adapt implicitly via merge_cost()
44 }
45 return sections;
46 }

```

Listing B.14: Example merge-cost weighting rules (illustrative, not final)

```

1 barrier: access control barriers are a merge cost of 1 (guaranteed separation), linear barriers
are mostly 0 cost
2 > block
3 > bollard
4 > cattle_grid
5 > chain
6 > cycle_barrier

```

```

7   > debris
8   > gate
9   > jersey_barrier
10  > log
11  > rope
12  > yes
13
14
15 Highway: change of highway type is a default merge cost of 0.5
16 Ranked from lowest to highest car popularity, the difference in the individual weighting of 2
    segments is multiplied with the default merge cost, giving a maximum of 1
17 > motorway -> motorway should never be hit since weighting is 0, but will always return a result
    of 1 since motorway is bad to go on
18 > trunk 1.9
19 > primary 1.8
20 > secondary 1.5
21 > tertiary 1.0
22 > unclassified 1.0
23 > road 1.0
24 > residential 0.5
25 > living street 0
26 the following highway types always incur a cost of 1 since link roads almost always mean changing
    roads
27 > motorway_link
28 > trunk_link
29 > primary_link
30 > secondary link
31 > tertiary link
32 > any misc road type: {bridleway, footway,service,track,path. etc}
33
34 cycleway: changing cycleway type has a merge cost of .2. Changing onto a cycleway (none to some,
    some to none) has a merge cost of 1
35
36 if highway has attribute junction, it has merge cost of 1
37
38 maxspeed: merge cost is the difference between 2 section's values * 0.02
39
40 mountain_pass = yes means merge cost of 1 (top of mountain)
41
42 smoothness: ranking from smoothest to least smooth. Difference * 0.05 for weighting
43 > excellent = 1
44 > good = .9
45 > intermediate = 0.7
46 > bad = 0.5
47 > horrible = 0.4
48 > very horrible = 0.2
49 > impassable -> automatic 1 merge cost
50
51 surface: going from paved-unpaved gives a merge score of .6, anything else gives score of .1

```


Appendix C

Application Documentation

Application Documentation

Pipeline Documentation for Thesis

September 8, 2025

Scope and purpose

This document describes the end-to-end data processing pipeline contained in `pipeline` directory. The main purpose of the pipeline is to take a GPS¹ track stored in the GPS Exchange file format (GPX), and produce a matched representation of that track on an OpenStreetMap road network. The result is an ordered sequence of matched points and metadata that can be visualised and further analysed. The documentation follows established guidelines for software user documentation as described in ISO/IEC/IEEE 26514². The pipeline is a component of the larger route segmentation system and is designed to be deployed as part of a web application backend. The code is already partly containerised using Docker, and the exposed scripts will eventually become HTTP endpoints when integrated into a web service. Recent development work has added native support for *batch processing*: instead of accepting only a single GPX file, the pipeline reads all files from a configurable source directory and processes them concurrently. This change means that multiple rides can be matched to the map in one run, and each input file produces its own matched output and visualisation. When ported to a web application, the same core code will be reused behind API endpoints that interact with the different pipeline stages. (Topografix 2023)

1 Overview of the pipeline

The pipeline is orchestrated by the Bash script `run_pipeline.sh`, contained in the `code/pipeline` directory. It reads settings from a YAML configuration file (`settings.yml`) and runs each component. The result of the pipeline is a matched GeoJSON route object that follows the path that the GPX file is suggesting. This removes GPS error from noise, and provides a structured network that can be used in analysis (Saki & Hagen 2022). Figures 1a to 1d show the comparison between mapped input GPX trackpoints and output GeoJSON route maps. The pipeline stages are listed below; see Figure 2 for a visual depiction. Each step logs its progress to the console, and the resulting artefacts are stored in the `data/` sub-directory.

¹GPS: Global Positioning System.

²<https://cdn.standards.iteh.ai/samples/77451/e161501288fd44f88c6fdd0f6e46b017/ISO-IEC-IEEE-26514-2022.pdf>

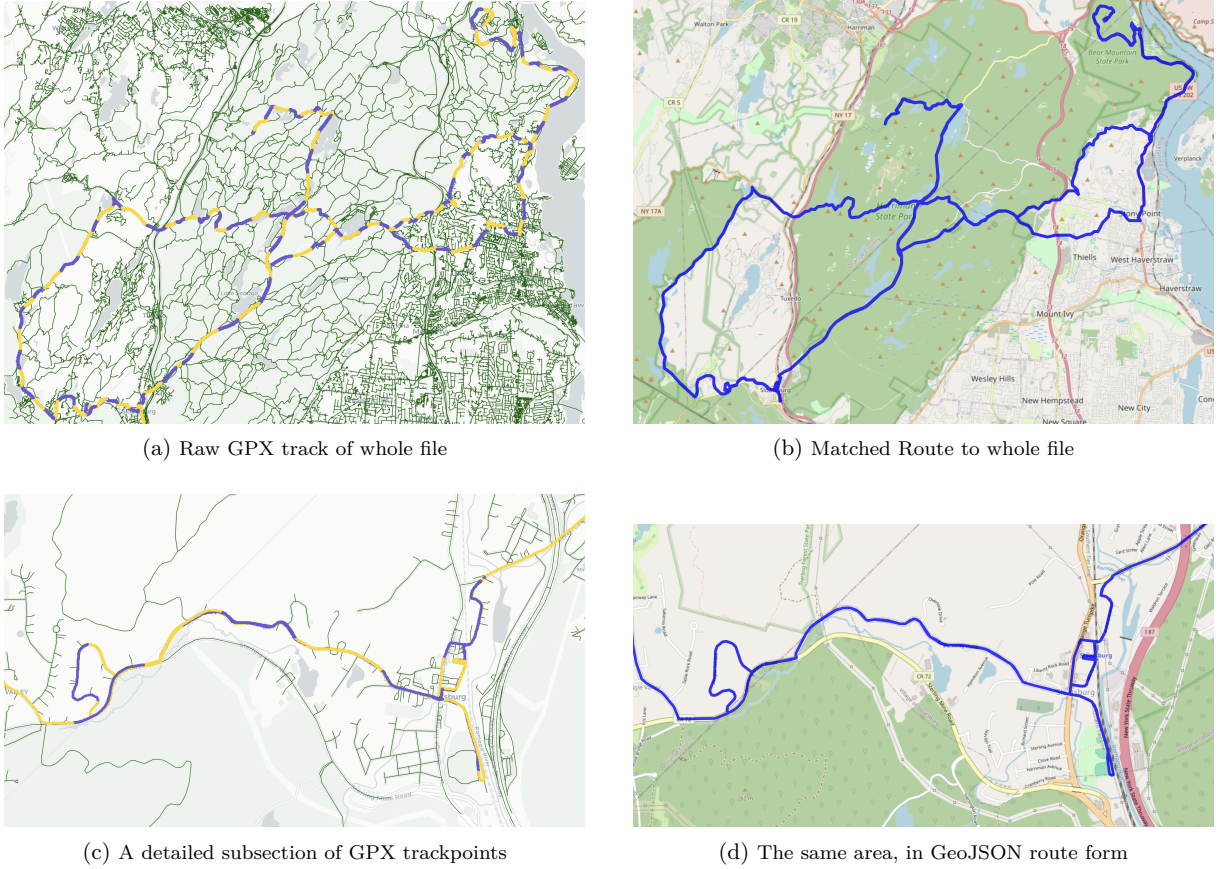


Figure 1: Comparison of raw GPX tracks and OSRM-matched routes: (a) GPX Track 1, (b) Route 1, (c) GPX Track 2, (d) Route 2.

1.1 Description of pipeline processing steps

1. **Configuration loading**—The script reads `settings.yml` to load initial settings for the pipeline. This includes (`gpx_source`) where the pipeline looks for `.gpx` files, and (`osm_files`) the directory where downloaded OpenStreetMap `.osm` files are stored. It also loads some other options, such as the size of the “chunks” (defined later) that the program splits each GPX file into.
2. **Environment setup**—A Python virtual environment is created (if it does not already exist) and dependencies from `requirements.txt` are installed. This isolates the pipeline from the system environment, and any future upgrades or dependency updates can be managed through this step.
3. **Data extraction and map download**—The shell script³ invokes the Python module `prepare_map.py`. In the current version the input to this module is a *directory* of GPX files. The script enumerates all `.gpx` files in the directory and, using a thread pool, performs the following steps for each file:
 - Parses the GPX track to obtain a sequence of latitude/longitude/timestamp points
 - Computes a bounding box around the full GPS track and downloads the OpenStreetMap `.osm` file for the bbox from the Overpass API, saving it as `<gpx_name>.osm` in the directory defined by `osm_files`. (Overpass API 2025)
 - Converts the track points into one or more OSRM GeoJSON objects and writes them to `data/temp/<gpx_name>`. Each JSON file represents a “chunk”⁴ of at most `chunk_size`

³The call is performed using Python’s `subprocess` module within `run_pipeline.sh`; for brevity the exact command is not reproduced here. See Appendix (Code Section) for the full shell script

⁴Chunking is necessary to avoid exceeding URL length limits when later calling the OSRM `/match` API, which accepts only HTTP GET requests.

points (a value of 0 in `settings.yml` disables chunking).

4. **Conversion of OpenStreetMap data to Protocolbuffer Binary Format (.pbf)**—Within a Docker container, the `osmium` tool converts the `.osm` files into the more compact `.pbf` format (Osmium Tool 2025). When multiple extracts have been downloaded (one per GPX file), they are merged into a single `merged.pbf` via the scripts located in `osmium/scripts`. The resulting file is renamed `map.pbf` and stored in `data/osrm_map/`, and the individual `.pbf` fragments are removed.
5. **Graph preparation with Docker**—Docker Compose is used to containerize the OSRM⁵ backend. The `osrm-prep` service preprocesses the `map.pbf` using the custom cycling profile in `docker/osrm/profile.lua` to build the routing graph using contraction hierarchies (Project OSRM 2025). Once preprocessing completes, the `osrm-server` service is launched in detached mode, exposing the `/match` endpoint on port 5000, ready for route matching.
6. **Route matching**—For each GPX file, the script invokes `batch_route_calc.py` on its corresponding chunk directory. This module reads the OSRM JSON chunks in their directories and performs parallel requests to the OSRM `/match` API using a thread pool. Each request uses a *dynamic radius algorithm* that increases the search radius per track point when the heading (direction) of the track has a high variance. This gives more context for `/match`'s algorithm to decide whether to turn off a road or not (Newson & Krumm 2009). The results of successful matches are saved as `result_*.json` files in `data/results/<gpx_name>`, while failures are saved to a `gap/` subdirectory for later gap filling using the `/route`⁶ endpoint.
7. **Merging**—Once all chunks for a GPX file have been processed, `merge_routes.py` recursively combines individual matching results into a single route. The merge logic decodes the geometries, concatenates all data arrays in the file, and computes aggregate distance and duration values. Merging is multi-threaded per recursion instance, and produces a single JSON file named after the input GPX file (e.g., `<gpx_name>.json`) saved directly in `data/results`. After merging, the per-file result chunks and their directories are removed.
8. **Visualisation**—Finally, `plot_merged.py` iterates over the merged JSON files in `data/results`, creating a Folium map for each route (Folium Developers 2025). The purpose of the map is mostly for debugging, as the main goal of the pipeline is to generate a continuous route that matches the GPX route as closely as possible. Each map can optionally overlay the raw GPS points for accuracy checking. The outputs are saved as `interactive_map_<gpx_name>.html` per input file, which can be opened in a web browser to inspect the matched route. This HTML can be served once the program is implemented into a web back-end, as a debug option.
9. **Clean up**—Temporary files (the intermediate JSON chunks, fragment `.pbf` files for individual GPX files and Docker volumes) are deleted. The virtual environment is deactivated and the OSRM containers are stopped and removed using Docker Compose. The script leaves the virtual environment directory intact so that subsequent runs reuse the existing dependencies.

⁵OSRM: Open Source Routing Machine.

⁶The `/route` endpoint calculates the path between two coordinates based on criteria such as distance or duration

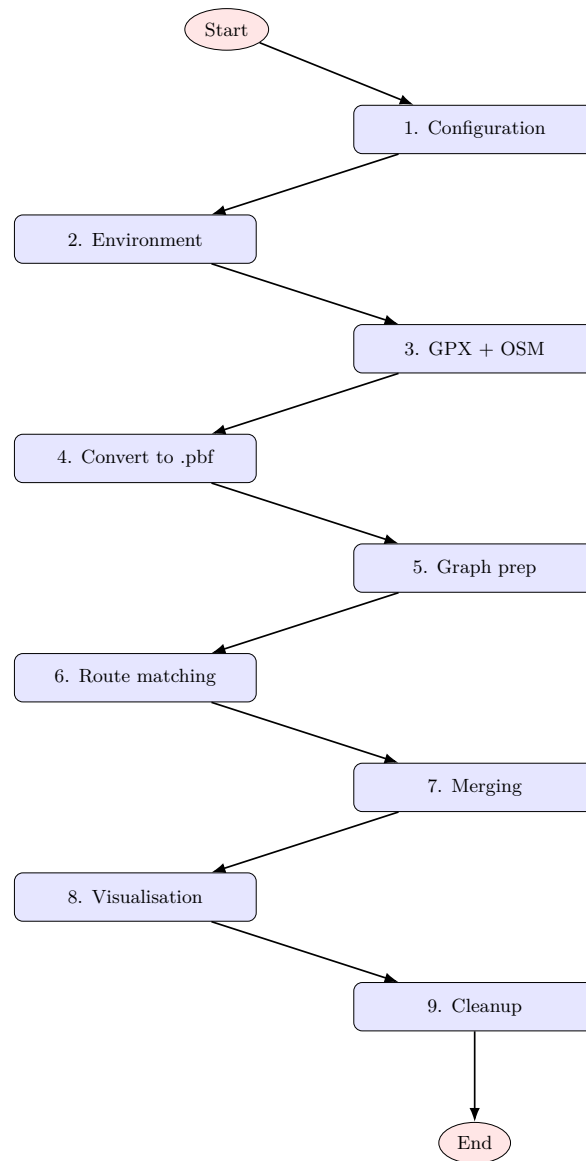


Figure 2: Pipeline flowchart for the GPX to OSRM-matched route processing system.

2 Python Module Breakdown

2.1 GPX Utilities (`gpx_utils.py`)

This module provides low-level helpers for GPX parsing, bounding-box computation, JSON I/O, and OSM downloads.

`load_gpx(path)` Opens and parses a GPX file using `gpxpy.parse()`, returning a GPX object with tracks, segments, and points. *Citation:* (Krajina 2023)

`extract_data_from_gpx(gpx)` Iterates through `gpx.tracks` $\rightarrow$ `.segments` $\rightarrow$ `.points`, building a list of dictionaries with:

- `lat, lon`: coordinates
- `elv`: elevation (or `None`)
- `time`: ISO-formatted timestamp

- any GPX `pt.extensions`, flattened into key-value pairs. Including but not limited to: Power, Cadence, Heartrate, and Temperature.

This list will be the heart of the categorization section of the program, as it contains all rider-specific metadata.

bounding_box_from_data(data) Given a list of point dicts, extracts all latitudes and longitudes, computes:

$$\text{south} = \min(\text{lats}), \quad \text{north} = \max(\text{lats}), \quad \text{west} = \min(\text{lons}), \quad \text{east} = \max(\text{lons}),$$

and returns the bbox tuple (south, west, north, east).

bbox_contains(outer, inner) Checks that the inner bbox lies fully within the outer by comparing numeric bounds:

$$\begin{aligned} \min(\text{lat}_{\text{inner}}) &\geq \min(\text{lat}_{\text{outer}}) \wedge \\ \min(\text{lon}_{\text{inner}}) &\geq \min(\text{lon}_{\text{outer}}) \wedge \\ \max(\text{lat}_{\text{inner}}) &\leq \max(\text{lat}_{\text{outer}}) \wedge \\ \max(\text{lon}_{\text{inner}}) &\leq \max(\text{lon}_{\text{outer}}) \end{aligned}$$

Returns True/False.

load_json(file_path) Reads and parses a JSON file via `json.load()`.

write_json(points, path) Ensures the target directory exists (`os.makedirs(..., exist_ok=True)`), then writes points with `json.dump(separators=(",", ":"), ensure_ascii=False)` for compact output.

download_osm_file(bbox, output_path) Constructs an Overpass “map” query URL of the form `https://overpass-api.de/api/map?bbox=west,south,east,north`, fetches via `requests.get()`, and writes the raw OSM XML to disk. *Citation:* (Overpass API 2025)

This module will also be used in the segmentation algorithm, but the relevant functions will be described in the Segmentation Algorithm documentation.

2.2 Data Extraction and Map Download (prepare_map.py)

The `prepare_map.py` module handles reading raw GPX tracks and fetching the corresponding OpenStreetMap extracts. It defines four primary functions:

parse_args() Uses Python’s `argparse` library to define and parse the command-line interface (Python Software Foundation 2025a).

- `gpx_file_path`: Input directory containing one or more `.gpx` files.
- `osm_output_path`: Directory in which to save downloaded `.osm` files.
- `radius`: Search radius (in metres) used for each track point.
- `chunk_size`: Maximum number of GPS points per OSRM JSON chunk (0 = no chunking).

thread_processes(gpx_file, output_path, chunk_dir, radius, chunk_size) Executed concurrently for each GPX file via `ThreadPoolExecutor` (Python Software Foundation 2025b):

1. **Load and parse GPX:** Calls `load_gpx(gpx_file)` and `extract_data_from_gpx(gpx)` from the `gpxpy` library to obtain a list of {lat, lon, timestamp} tuples (Krajina 2023).
2. **Prepare output paths:** Derives `gpx_name` via `os.path.splitext` and ensures `output_path` exists using Python’s `os` module (Python Software Foundation 2025c).
3. **Chunk for OSRM:** Invokes `points_to_osrm_json(points, chunk_dir, gpx_name, radius, chunk_size)` (in `utils/osrm_utils.py`) to split the track into one or more JSON payloads ready for OSRM’s `/match` endpoint.
4. **Fetch OSM if needed:** If `<gpx_name>.osm` does not already exist under `output_path`, calls `get_single_osm_from_gpx()`, otherwise logs a warning and skips.

get_single_osm_from_gpx(gpx_file, points, gpx_name, output_path) Retrieves OSM data via Overpass API:

- **Compute bounding box:** Uses `bounding_box_from_data(points)` (from `utils/gpx_utils.py`) to derive the min/max latitude and longitude.
- **(Re)load GPX points:** Calls `load_gpx()` and `extract_data_from_gpx()` again to ensure consistency.
- **Download OSM file:** Invokes `download_osm_file(bbox, full_output_path)`, which constructs an Overpass API query and writes the resulting `.osm` XML to disk.

main() Orchestrates the module workflow:

- Parses CLI arguments via `parse_args()`.
- Scans `gpx_file_path` for `.gpx` files using `os.listdir()` and `os.path.isfile()` (Python Software Foundation 2025c).
- Determines `MAX_THREADS` as `min(32, number of GPX files)`.
- Creates a `ThreadPoolExecutor(max_workers=MAX_THREADS)` and submits `thread_processes()` tasks for each GPX file.
- Uses `as_completed()` to wait for all threads to finish before exiting.

This design parallelizes I/O and network operations, which are the most time-costly operations.

3 Configuration options

The pipeline reads configuration values from `settings.yml`. Table 1 summarises the available keys and their meanings. Ensure that file paths refer to existing locations and that chunk sizes remain within reasonable limits to avoid exceeding URL length limits when calling the OSRM API.

Table 1: Summary of configuration keys in `settings.yml`.

Key	Description
<code>gpx_source</code>	Directory containing one or more GPX files. All files with the <code>.gpx</code> extension in this directory will be processed. Example: <code>data/input</code> .

Continued on next page

Table 1 continued from previous page

Key	Description
<code>chunk_size</code>	Maximum number of points per OSRM JSON file. A value of 0 disables chunking. Large values may exceed the URL length limit of the OSRM API; the script enforces a maximum of 200.
<code>radius</code>	Initial search radius (in metres) used when matching points to the road network. The dynamic radius algorithm may adjust this value on a per-point basis. Example: 5.
<code>dynamic_radius_window</code>	Size of the initial sliding window used when computing dynamic radii. Larger values smooth the radius changes over more points; smaller values react more quickly to heading changes.

4 Output JSON file structure

When the OSRM map-matching step completes successfully, each GPX track chunk is written to disk as a JSON file (`result_*.json` in `data/results/<gpx_name>`). After all chunks for a file have been matched, `merge_routes.py` concatenates the results into a single JSON object named after the input file (e.g. `<gpx_name>.json`). Understanding the structure of these JSON files is important for downstream analysis and for the enrichment step that augments them with sensor data. This section summarises the key top-level keys and nested objects you will encounter.

4.1 Top-level keys

- **code** – A short status string returned by OSRM. A value of `"Ok"` means that all points in the input chunk were matched successfully. If matching fails for any point, the code may contain error descriptions (e.g. `"NoSegment"`).
- **tracepoints** – An array with one entry per input point. Each element is either a JSON object describing the matched road coordinate or `null` if the point could not be matched.
- **matchings** – A list of matched routes. Although the pipeline normally produces one matching per chunk, OSRM may occasionally split the route into multiple matchings when long gaps exist. Each matching contains aggregated route metrics and the geometry of the matched path.

4.2 Tracepoints and GPX enrichment

The `tracepoints` array aligns one-for-one with the original coordinates sent to OSRM. For a matched point the tracepoint object has the form

```
{
  "location": [lon, lat],
  "name": "",           // optional road name or empty string
  "matchings_index": i, // index of the matching this point belongs to
  "waypoint_index": j,  // position of this point in the matching
  "alternatives_count": k, // number of alternative candidates considered
  "classes": [ ... ],   // optional OSM road classes

  "gpx_points": [{
    "time": "YYYY-MM-DDTHH:MM:SSZ",
    "elv": h,           // elevation in metres or null
    "extensions": {
```

```

    "hr_bpm": <heart rate>,
    "cad_rpm": <cadence>,
    "power_w": <power>,
    "speed_mps": <speed>,
    "temp_c": <temperature>,
    "alt_m": <altitude>,
    "grade_pct": <road grade>,
    ...
  }
}, ... // multiple gpx points per tracepoint. contains points up until the next tracepoint.
]
}
```

The nested `gpx` object is an addition introduced in this project. It stores metadata extracted from the original GPX file:

- `time` and `elv` carry the timestamp and elevation of the tracepoint.
- `extensions` is a dictionary of canonical sensor values normalised according to `config/extensions_map.json`. The canonical keys (`hr_bpm`, `cad_rpm`, `power_w`, `speed_mps`, `temp_c`, `alt_m` and `grade_pct`) are derived from the raw GPX extension tags via unit conversion and type coercion. Missing values are omitted from this dictionary.

Null entries in the `tracepoints` array indicate gaps where OSRM could not match a GPS coordinate to the network. These are carried through to the merged file so that you can identify portions of the track that require gap-filling via the `/route` API.

4.3 Matchings and their legs

Each entry in the `matchings` list represents a continuous segment of the route. A matching has the following important members:

- **confidence** – A floating-point value in $[0, 1]$ that quantifies OSRM's certainty that the matching is correct. Values close to 1 indicate that the GPS data fitted the road network well.
- **distance** and **duration** – Aggregate length (in metres) and travel time (in seconds) of the matching. These are sums over all legs.
- **geometry** – An encoded polyline string representing the path of the matching. It can be decoded into a list of coordinates using standard polyline libraries.
- **legs** – A list of legs; one leg for each pair of consecutive waypoints. Legs contain fine-grained information such as the line geometry between two matched points, a summary with per-leg distance and duration, and optionally annotations describing speed and elevation. When the pipeline merges results the leg lists from all chunks are concatenated.

Each leg has the optional datapoint "annotation", so that each node ID can be pulled, and matched against the SQLite indexed database (SQLite 2025) to get the respective way ID. The way IDs are then implemented into the leg object in the form of "runs". Each run is a semi-hashed list of tuples: (`way_id`, `direction`, `edge_count`), where the `direction` is either 1 or -1, depending on if the match in the database is forward or backwards. The `edge_count` variable counts the number of consecutive nodes that fall along the way. ?? shows a visual representation of each leg, with each colour change representing a new leg.

Construction

1. Read `leg.annotation.nodes` and form consecutive pairs.

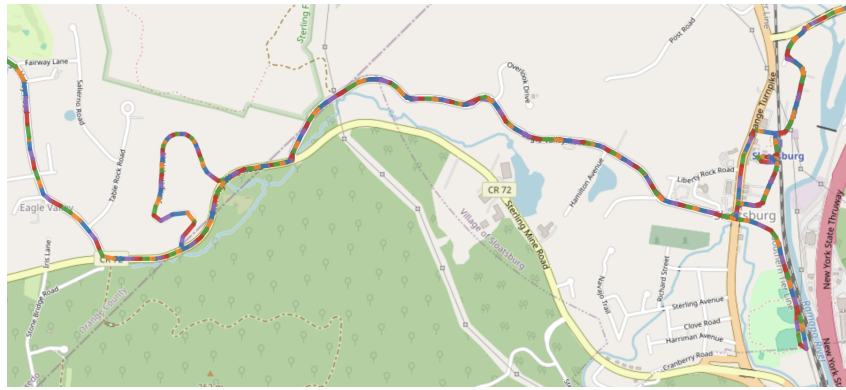


Figure 3: Visual representation of leg segments

2. For each pair, look up the corresponding edge in the SQLite way index(SQLite 2025) to obtain (way_id, dir, length).
3. Group adjacent pairs with the same (way_id, dir) and emit one object per group.

Fields in each run

- way_id — OpenStreetMap way identifier.
- dir — travel direction relative to the way's stored orientation (e.g., 1 or -1).
- edge_count — number of node-to-node edges collapsed into the run.
- length_m (optional) — total length of the run in metres (present when run with --with-metrics).

```
{
  "annotation":{
    "nodes":[                // ordered list of node IDs the leg travels through
      0123456789,
      1234567890,
      ...
    ],
    "distance":[              // Per-edge length (in metres) between consecutive nodes
      15.385,
      8.0151234,
      ...
    ]
  },
  "distance":29.8,            // total leg length in metres
  "duration":2.3,             // OSRM-calculated travel time from weightings in profile.lua
  "weight":2.3,               // OSRM cost for the leg. Is equal to duration in our case
  "runs":[                    // the pipeline compression of consecutive edges
    {
      "way_id": 98765432,     // Way that the path is on
      "dir": 1,               // direction of travel, 1 is forwards, -1 is backwards
      "edge_count":5          // the run covers 5 edges in the way
    }, ...
  ],
  "total_edges":17            // Number of node-to-node edges inside the leg..
}, ...
```

Downstream modules (e.g. the segmentation and analysis code) can iterate over `matchings` and their `legs` to compute cumulative distance, extract elevation profiles, or associate sensor readings with segments of the road. The enrichment process described earlier ensures that each tracepoint contains not only its matched map location but also the underlying GPX metadata for comprehensive analysis.

5 Running the pipeline

This section provides a step-by-step procedure for executing the pipeline. It assumes that both Docker and Python 3.10 or higher are installed on the host system.

1. **Clone the repository** if you have not already done so. In a terminal, run:

```
git clone https://github.com/jibb34/MScProject && cd MScProject/code/pipeline
```

This will put you into the main pipeline directory, and you can see all the files and subdirectories with: `ls -la .`
2. **Inspect and modify `settings.yml`** to set `gpx_source` to the directory containing your GPX files and adjust parameters as needed. Do not exceed a chunk size of 200 points. If necessary, increase the search radius if the file contains a lot of GPS noise
3. **Execute the pipeline** by running the shell script:

```
bash ./run_pipeline.sh
```

The script will create a Python virtual environment, install dependencies, download the map, build the OSRM graph and perform route matching. Progress messages are printed to the console. While the script is running, do not attempt to modify any files or paths in the main `pipeline` directory

4. **View the results.** After the script completes, open all `interactive_map_<gpx_name>.html` files (one file per input GPX) in a web browser. The file is located in the main directory. You can also inspect the merged JSON files for a structured representation of the matched route. They are located in `./data/results`
5. **Clean up.** The script performs clean-up automatically. Temporary files are removed and containers are shut down. If you wish to start a clean run, you may delete the `.venv` directory to remove the python environment, and you should delete all html files and GeoJSON files with this command:

```
rm -rf .venv ./data/results/* .//*.html
```

6 Error handling and warnings

The pipeline contains the following safeguards:

- If `settings.yml` is missing or lacks required keys, the script exits immediately with an error message.
- The chunk size is validated to ensure it does not exceed 200 points. Exceeding this limit may produce URLs that are too long for the OSRM server to process; a fatal error is raised if the limit is broken.
- Requests to the OSRM server are wrapped in exception handling. If a request fails (e.g., network error), an empty result is produced and saved to a `gap/` directory for later gap filling. This will be implemented later for more robust behaviour if necessary. (So far, no empty requests have occurred outside of programmatic errors.)
- During merging, the script checks for missing geometries or mismatched code values and reports errors if encountered. Merging proceeds even when some segments cannot be combined, but is flagged as incomplete.

- If the Folium plotting script cannot find any match result files, it raises an explicit `FileNotFoundError`.

References

- Folium Developers (2025), ‘Folium: Python library for interactive maps’, <https://folium.readthedocs.io>. Accessed August 2025.
- Krajina, T. (2023), ‘*gpxpy* (version 1.6.2) [python library]’, <https://pypi.org/project/gpxpy/>. Accessed August 2025.
- Newson, P. & Krumm, J. (2009), Hidden markov map matching through noise and sparseness, in ‘Proceedings of the 17th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS’09)’, Seattle, WA, USA.
- Osmium Tool (2025), ‘Multi-purpose osm data manipulation toolkit’, <https://osmcode.org>. Accessed August 2025.
- Overpass API (2025), ‘Openstreetmap data extractor’, <https://overpass-api.de>. Accessed August 2025.
- Project OSRM (2025), ‘Open source routing machine (osrm)’, <http://project-osrm.org>. Accessed August 2025.
- Python Software Foundation (2025a), ‘argparse — parser for command-line options (python documentation)’, <https://docs.python.org/3/library/argparse.html>. Accessed August 2025.
- Python Software Foundation (2025b), ‘concurrent.futures — launching parallel tasks (python documentation)’, <https://docs.python.org/3/library/concurrent.futures.html>. Accessed August 2025.
- Python Software Foundation (2025c), ‘os — miscellaneous operating system interfaces (python documentation)’, <https://docs.python.org/3/library/os.html>. Accessed August 2025.
- Saki, S. & Hagen, T. (2022), ‘A practical guide to an open-source map-matching approach for big gps data’, *SN Computer Science* **3**(415).
- SQLite (2025), ‘Sqlite home page’, <https://sqlite.org>. Accessed August 2025.
- Topografix (2023), ‘Gpx: the gps exchange format (version 1.1)’, <http://www.topografix.com/gpx.asp>. Accessed August 2025.

Segmentation Engine Documentation

September 8, 2025

1 Scope

This document describes the Segmentation Engine, a C++ HTTP service for route analysis and segment extraction. It has been prepared in accordance with ISO/IEC/IEEE 26514:2008, which defines structure and content for software user documentation.

2 Normative References

- ISO/IEC/IEEE 26514:2008 – Systems and software engineering – Requirements for designers and developers of user documentation.
- ISO 8601:2019 – Date and time representation.

3 Terms and Definitions

Segment A contiguous portion of a route, represented as GeoJSON geometry and associated metadata.

Route signal A sampled representation of map variables (elevation, curvature, ...) indexed by distance along the route.

Bounding box Geographic region defined by west, south, east and north coordinates.

4 System Overview

The engine exposes a RESTful API that receives map data, builds route signals, performs wavelet based segmentation and persists results to a MySQL database. The major modules are:

- **core**: algorithmic components such as the route signal builder, segmentation logic and wavelet utilities.
- **http**: request routing and REST endpoint handlers built on `cpp-httplib`.
- **infra**: database adapter for MySQL persistence.
- **models**: C++ types representing OSRM responses, segments and configuration parameters.

5 Installation and Operation

5.1 Prerequisites

CMake 3.15 or newer, a C++17 capable compiler, Docker and a running MySQL server.

5.2 Build and Deployment

The provided script builds and runs the engine inside a Docker container:

```
./build_and_deploy.sh
```

It performs three steps:

1. Build an image tagged `segmentation_engine:latest`.
2. Remove any existing container named `segmentation_container`.
3. Launch a new container on port 5005 with MySQL parameters (`DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASS`, `DB_NAME`) supplied via the environment.

Internally, the script issues the following Docker commands:

```
docker build -t segmentation_engine:latest .
docker rm -f segmentation_container >/dev/null 2>&1 || true
docker run -d --name segmentation_container --network host \
  -e DB_HOST=127.0.0.1 -e DB_PORT=3306 -e DB_USER=routeseq_user \
  -e DB_PASS=changeme-user -e DB_NAME=routeseq -p 5005:5005 \
  segmentation_engine:latest
```

Container logs can be monitored via:

```
docker logs -f segmentation_container
```

For a manual build without Docker:

```
cmake -S . -B build
cmake --build build
./build/segmentation_engine
```

5.3 Configuration

Runtime settings, including the HTTP port and enabled endpoints, are stored in `config/settings.json`. Database credentials are compiled into `src/main.cpp` and should be configured prior to deployment.

5.4 Execution

After deployment the service is reachable on `http://localhost:5005` and mounts the `public` directory under `/static`. POST and GET routes are registered from the configuration file.

To consolidate endpoints and avoid Cross-Origin Resource Sharing (CORS) issues, the parent `segmentation` directory provides a lightweight Nginx proxy. From that directory run:

```
./proxy.sh
```

The script starts a container named `quick-proxy` that forwards port 8081 to the Segmentation Engine at 5005, exposing the service at `http://localhost:8081`. Additional proxy ports 8082 and 8083 target the viewer and database UI, allowing browser clients to interact with all components through a single origin.

6 REST API

Table 1 lists the available endpoints. JSON bodies use UTF-8 encoding and timestamps follow ISO 8601.

Table 1: REST endpoints

Method/Path	Description
POST /upload	Accepts a map JSON file and stores it under <code>uploads/</code> . Returns the storage path.
GET /view?map= <i>path</i>	Renders an interactive HTML viewer comparing raw OSRM geometry to the derived route signal.
GET /viewLab?map= <i>path</i>	Redirects to the Signal Lab user interface.
GET /lab/meta?map= <i>path</i>	Lists available uploads and optionally returns metadata for a specific map.
POST /lab/resample	Builds a route signal and resamples requested variables onto a uniform distance grid.
POST /wavelet	Performs wavelet-based terrain segmentation and optionally persists segments to the database.
GET /segments	Retrieves previously persisted segments intersecting a geographic bounding box.

7 Algorithms

Segmentation is accomplished by constructing a uniformly sampled route signal from the uploaded map and applying a wavelet-based energy detector. High energy regions correspond to changes in elevation or curvature and are converted into segments. Utilities in `core/wavelets` implement the FFT based rolling energy calculation. Existing segments can be matched against new routes to avoid redundant processing.

8 Data Management

Segment metadata is stored in a MySQL schema defined by `schema.sql`. The `infra::MySQLSegmentDB` component manages connections, inserts new segments and queries by bounding box.

9 Error Handling and Logging

Fatal signals (segmentation faults, aborts, arithmetic errors) are trapped and a stack trace is printed to the server log. Endpoint handlers translate exceptions into HTTP 500 responses containing a diagnostic message.

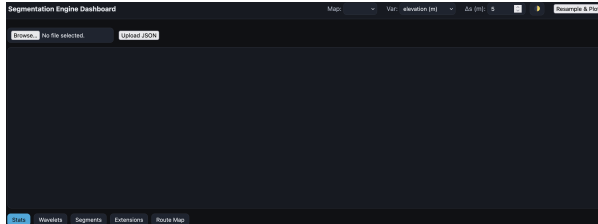
10 Conformance

The Segmentation Engine and this document use ISO 8601 timestamps and the terminology defined in ISO/IEC/IEEE 26514:2008. All numeric values are expressed using SI units.

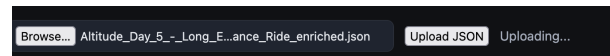
11 Application Screenshots

11.1 Dashboard and Data Upload

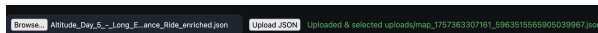
In line with ISO/IEC/IEEE 26514:2008 guidance on task-oriented documentation, this sequence demonstrates the primary user workflow: initial dashboard state, file transfer progression, confirmation of successful ingestion, and verification via route–OSRM alignment. The figure substantiates that the ingest–validate loop is operational end to end.



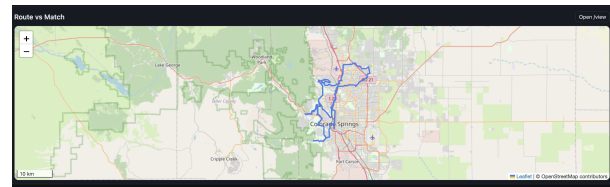
(a) Default dashboard on initial load.



(b) Upload in progress.



(c) Upload complete.



(d) Route vs matched geometry.

Figure 1: Loading, uploading, and path matching workflow.

11.2 Database Viewer

This view evidences persistence and retrieval capabilities as required by the standard’s “data management and storage” descriptions. It shows that extracted segments are indexed, queryable, and auditable, supporting reproducibility of results and post hoc analysis.

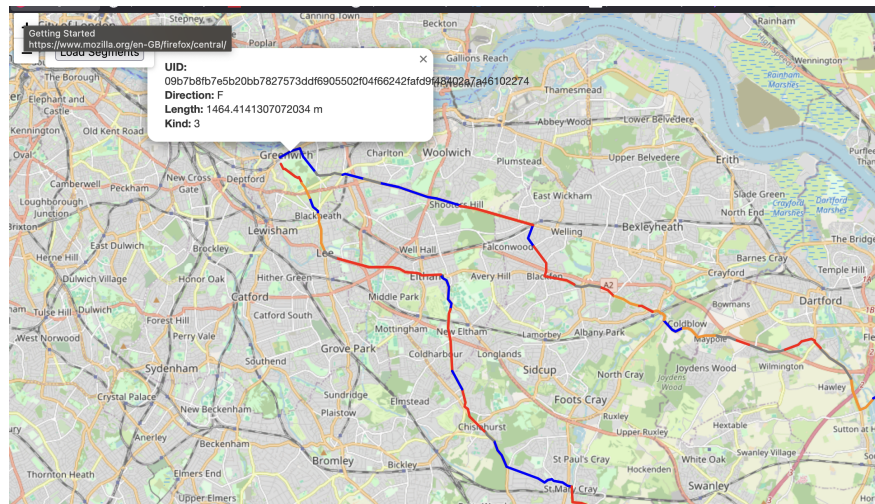
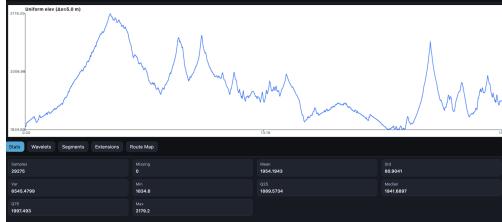


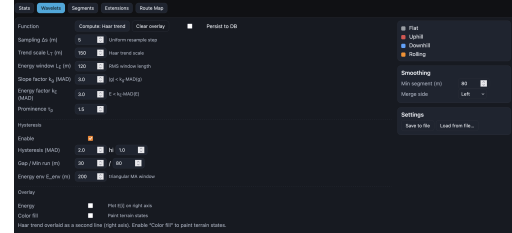
Figure 2: Interactive segment database inspector.

11.3 Signal and Wavelet Panels

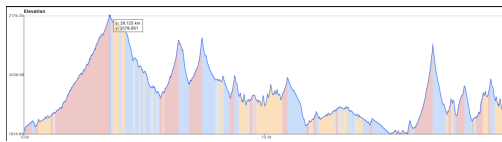
These panels expose algorithm configuration and outputs, satisfying the standard's expectation for clarity of computational steps. Elevation-versus-distance, tunable wavelet parameters, and the scalogram together justify threshold and segmentation choices, enabling readers to trace input settings to resultant segments.



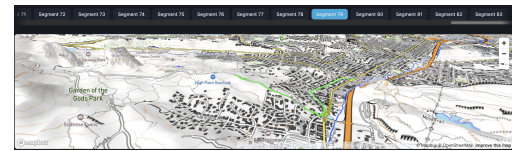
(a) Elevation view ($\Delta s = 5\text{m}$).



(b) Wavelet controls.



(c) Scalogram with trend overlay.



(d) Segmentation options.

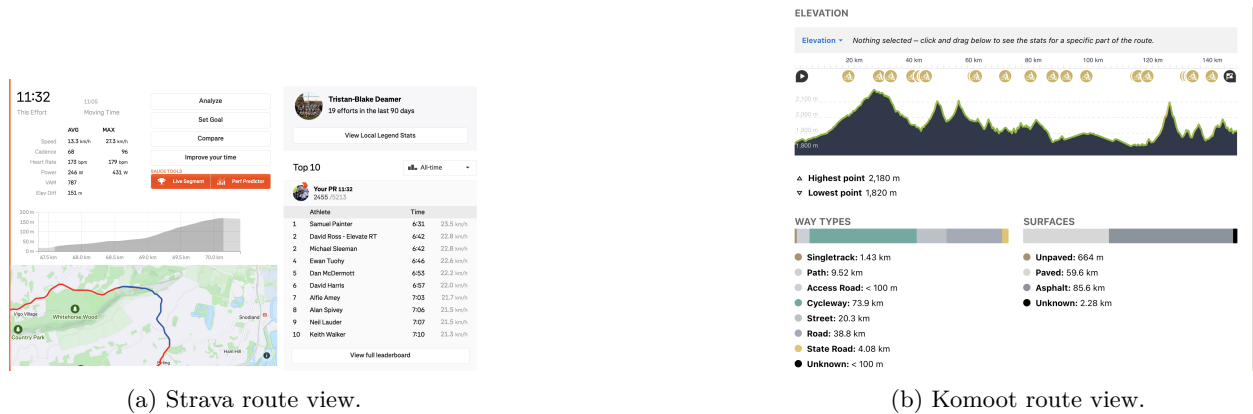
Figure 3: Signal plots, wavelet parameters, and segmentation settings.

12 Extensions Graph



Figure 4: Auxiliary signals (extensions) aligned to distance.

12.1 External Platform Views



(a) Strava route view.

(b) Komoot route view.

Figure 5: Comparison: Strava vs Komoot.

Appendix D

Glossary

GPX GPS Exchange Format used to encode routes and activities.

GeoJSON JSON-based format for geographic data structures.

OSM OpenStreetMap, the open collaborative map database.

OSRM Open Source Routing Machine providing map matching services.

TSS Training Suitability Score assigned to each segment.

ERD Entity–Relationship Diagram illustrating database structure.

DTO Data Transfer Object used for moving data between modules.